Sistema terza missione

Francesco Carmine

1 Descrizione del problema

1.1 Analisi e specifica dei requisiti

RF1 - Gestione Utenza e Sicurezza:

RF1.1- Autenticazione: Il sistema deve permettere l'accesso a Docenti, Responsabili ed Amministrazione avvalendosi esclusivamente delle identità istituzionali di Ateneo. Non è prevista una funzione di registrazione pubblica indipendente; l'accesso è vincolato agli utenti già censiti dall'anagrafica universitaria.

RF1.2 – **Autorizzazione per Ruolo**: Ogni ruolo può eseguire solo le operazioni consentite dal proprio profilo. Il sistema deve distinguere tra ruoli assegnati (persistenti) e ruolo attivo (nella sessione corrente), permettendo all'utente di effettuare il cambio ruolo (switch) senza dover effettuare nuovamente il login.

RF1.3 – Gestione Nomine (Responsabile): I profili utente di base (Docente, Personale Amministrativo) sono acquisiti automaticamente dal sistema di Ateneo. Il ruolo di Responsabile di Struttura è gestito tramite nomina temporanea (con data inizio e fine). Il sistema rimuove automaticamente i privilegi alla scadenza naturale del mandato. L'Amministrazione può effettuare una revoca anticipata manuale (es. per dimissioni o pensionamento) terminando l'incarico prima della scadenza prevista.

RF2 - Gestione Attività: Il sistema deve permettere al Docente di creare, modificare (in stato di bozza) e cancellare le proprie attività (CRUD)

RF3 - Inserimento Dati: Il sistema deve permettere l'inserimento di Titolo, Descrizione, Tipologia, Data di inizio e di fine, ed eventuali Allegati. Se la tipologia selezionata è "Divulgazione e Public Engagement", il sistema deve **prevedere un campo** per indicare il numero di partecipanti. Gli allegati devono rispettare i seguenti vincoli tecnici:

Formati e Dimensione: Sono ammessi esclusivamente formati standard (PDF, JPG, PNG) con dimensione massima di 5MB per file.

Sicurezza e Storage: I file sono sottoposti a scansione antivirus all'upload e salvati su File System protetto (il database memorizza esclusivamente il percorso del file).

Retention Policy: In ottemperanza al GDPR, i file allegati vengono eliminati fisicamente dal server contestualmente alla cancellazione dell'attività associata (o al massimo entro 30 giorni dalla stessa per scopi di backup).

RF4 - Workflow di Approvazione: Il sistema deve gestire le transizioni di stato applicando le seguenti autorizzazioni:

Bozza -> Inviata: Eseguibile esclusivamente dal Docente (autore dell'attività).

Inviata -> Approvata: Eseguibile esclusivamente dal Responsabile di Struttura (o rifiutata, retrocedendo in Bozza).

Approvata -> Validata: Eseguibile esclusivamente dall'Amministrazione (o rifiutata, retrocedendo in Bozza).

RF4.2 – Conflitti d'interesse: Il sistema deve impedire che un utente approvi un'attività di cui è egli stesso l'autore, anche qualora possieda i permessi di Responsabile.

RF5 – Consultazione e permessi: Il sistema deve regolare la visibilità delle attività in base al Ruolo Attivo e allo Stato dell'attività, secondo le seguenti regole:

Docente (Autore): Ha visibilità esclusiva sulle proprie attività in stato di "Bozza". Mantiene la visibilità in sola lettura per tutti gli stati successivi**.**

Responsabile di Struttura: Ha visibilità sulle attività dei docenti afferenti alla propria struttura, ma esclusivamente quando si trovano negli stati "Inviata", "Approvata" o "Validata". l Responsabile non può assolutamente visualizzare le attività in stato di "Bozza" degli altri utenti.

Amministrazione: Ha un permesso di visibilità globale su tutto l'Ateneo (indipendentemente dalla struttura), ma limitato alle sole attività che hanno superato il primo step di controllo (stati **"**Approvata" e "Validata**"**).

RF6 - Reporting e Analisi Dati:

RF6.1 – Generazione: Il sistema deve permettere all'Amministrazione di generare report statistici basati esclusivamente sulle attività in stato "Validata". L'utente deve poter filtrare i dati per: **Anno Accademico** (o intervallo temporale). **Dipartimento del docente**. **Tipologia di Attività** (Pubblicazione, Convegno, Terza Missione). Il sistema deve mostrare immediatamente a video un'anteprima sintetica dei risultati.

RF6.2 – Storicizzazione (Audit): Il sistema deve memorizzare permanentemente nel database l'avvenuta generazione di ogni report. Per ogni operazione di reporting devono essere salvati: **Autore:** Chi ha richiesto il report. **Data e Ora:** Timestamp della generazione. **Filtri utilizzati**. **Dati sintetici**.

RF6.3 – Esportazione Locale: A partire da un report generato (o recuperato dallo storico), il sistema deve permettere l'esportazione dei dati in formati standard. l file viene salvato in locale sulla macchina dell'utente tramite finestra di dialogo di sistema.

RF7 – Vincoli di Modifica: Il sistema deve impedire la modifica e cancellazione delle attività che non si trovano nello stato di "Bozza". Le attività in stato "Inviata", "Approvata" o "Validata" devono essere in sola lettura per il Docente.

RF8 – Gestione Rifiuto: Il Responsabile e l'Amministrazione devono poter rifiutare un'attività (facendola retrocedere di stato, es. a Bozza) inserendo obbligatoriamente una motivazione testuale visibile al docente. Il sistema deve garantire la storicizzazione permanente di ogni rifiuto, memorizzando per ogni feedback i seguenti dati obbligatori:

Autore: Identificativo dell'utente che ha emesso il rifiuto (Responsabile o Amministrazione).

Timestamp: Data e ora precisa dell'operazione.

Esito: esito della revisione.

Motivazione: Testo descrittivo della causa del rifiuto

Tali dati devono restare consultabili nella storia dell'attività anche in seguito a modifiche successive

RF9 – Visibilità per Struttura: Il sistema deve filtrare le attività visibili al Responsabile, mostrandogli solo quelle create dai Docenti afferenti alla sua stessa struttura (Dipartimento).

RF10 – Gestione Notifiche: Il sistema deve inviare notifiche automatiche tramite duplice canale (Email istituzionale e Avviso In-App) al verificarsi dei seguenti eventi:

Al **Responsabile di Struttura**: Quando un docente invia una nuova attività (passaggio a stato "Inviata") che richiede la sua approvazione.

Al **Docente** (Autore): Quando una sua attività viene Approvata, Validata o Rifiutata. Nel caso di rifiuto, la notifica deve contenere un estratto della motivazione (feedback).

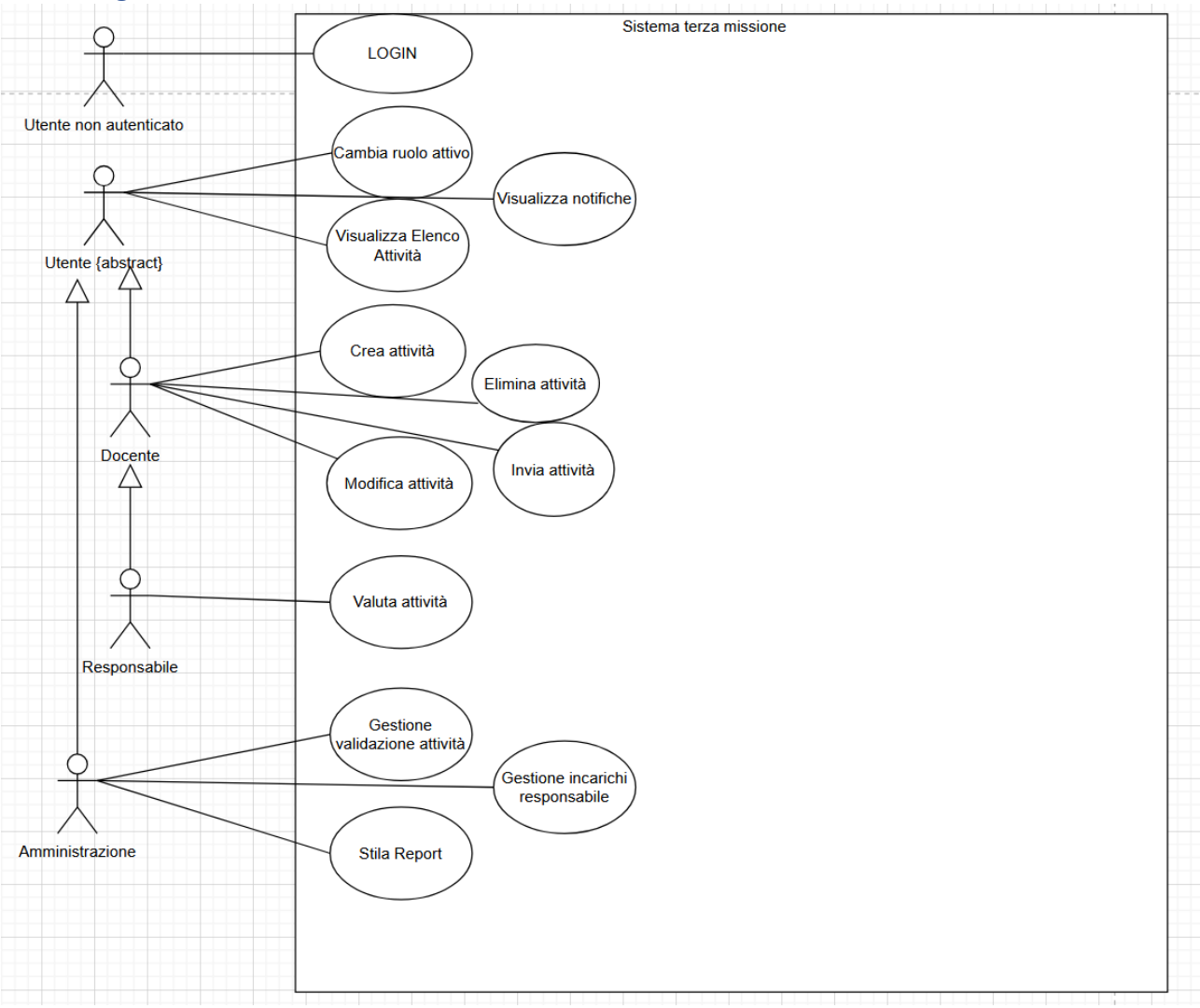
RNF1 - Persistenza: I dati devono essere memorizzati su database relazionale (es. PostgreSQL/MySQL).
RNF2 - Interfaccia: La GUI deve essere sviluppata in Java (JavaFX).
RNF3 - Portabilità: Il sistema deve essere eseguibile su qualsiasi piattaforma che supporti la JVM.
RNF4 – Sicurezza Dati: Le password degli utenti devono essere memorizzate tramite hash crittografico e non in chiaro.
RNF5 – Architettura a Livelli: Il sistema deve essere progettato separando nettamente la logica di presentazione (View), la logica di business (Controller/Service) e l'accesso ai dati (DAO/Model), implementando il pattern MVC
RNF6 - Sincronizzazione e Integrità Dati: Per garantire elevate prestazioni applicative e supportare i vincoli referenziali, il sistema deve mantenere nel proprio database una copia locale dei dati anagrafici essenziali. Le credenziali vengono sincronizzate automaticamente con l'Identity Provider di Ateneo.

1.2 Glossario

Afferenza: Il legame amministrativo che associa un Docente a una specifica struttura (Dipartimento). Determina quale Responsabile ha visibilità sulle attività del Docente.
Amministrazione: Personale tecnico-amministrativo incaricato della validazione formale delle attività approvate ai fini della reportistica ufficiale
Attività di Terza Missione: Insieme delle attività di trasferimento scientifico, tecnologico e culturale e di trasformazione produttiva delle conoscenze
Bozza: Stato iniziale di un'attività inserita nel sistema, visibile e modificabile solo dal docente proponente
Docente: Utente accreditato che inserisce, modifica e sottopone ad approvazione le attività di Terza Missione
Feedback (o Motivazione): Testo obbligatorio inserito dal Responsabile o dall'Amministrazione in caso di rifiuto di un'attività, storicizzato dal sistema per garantire la tracciabilità della decisione.
Responsabile di Struttura: Docente con ruolo di supervisione (es. Direttore di Dipartimento) incaricato di approvare i contenuti inseriti dai docenti afferenti alla propria struttura.
Ruolo Attivo: Il profilo con cui l'utente sta operando nella sessione corrente (es. un Direttore può scegliere di operare come "Docente" per inserire le proprie attività o come "Responsabile" per approvare quelle altrui).
Validazione: Passaggio conclusivo del ciclo di vita dell'attività, eseguito dall'Amministrazione, che rende l'attività idonea per il report annuale.

2 Progettazione del Sistema

2.1 Diagramma dei casi d'uso



2.2 Descrizione degli scenari

Nome	**Login**
Scopo	Permettere all'utente di accedere al sistema e identificare il proprio ruolo
Attore/i	Utente non autenticato
Pre-condizioni	L'utente possiede già le credenziali istituzionali ed è censito nel database
Trigger	L'utente avvia l'applicazione
Descrizione sequenza eventi	1. Il sistema mostra la schermata di login 2. L'utente inserisce le credenziali istituzionali e seleziona il ruolo con cui vuole operare 3. L'utente preme accedi 4. Il sistema interroga il database locale per verificare la corrispondenza delle credenziali fornite, confrontando la password inserita con l'hash crittografico salvato. 5. Il sistema verifica che l'utente sia effettivamente abilitato e in possesso del Ruolo selezionato.

	6. Il sistema inizializza la sessione utente con quel ruolo attivo. 7. Il sistema reindirizza l'utente e mostra la dashboard principale specifica per quel ruolo.
Alternativa/e	4b:**Credenziali errate**: Se la query sul database non trova corrispondenze valide, il sistema mostra un messaggio di errore 5b **Ruolo non autorizzato**: l'utente ha selezionato un ruolo che non possiede. Il sistema mostra: "Accesso negato: non possiedi i privilegi per il ruolo selezionato".
Post-condizioni	L'utente è autenticato e il Ruolo Attivo corrisponde a quello scelto in fase di login

Nome	**Cambia Ruolo Attivo**
Scopo	Permettere all'utente di cambiare il proprio ruolo attivo per accedere a funzionalità diverse senza effettuare il logout.
Attore/i	Utente autenticato
Pre-condizioni	L'utente è autenticato e al suo profilo sono associati almeno due ruoli validi (es. è sia Docente che Responsabile).
Trigger	L'utente clicca su Cambia Ruolo
Descrizione sequenza eventi	1. Il sistema mostra l'elenco dei ruoli disponibili per l'utente 2. Il sistema evidenzia il ruolo attualmente attivo. 3. L'utente seleziona il nuovo ruolo desiderato. 4. Il sistema aggiorna i permessi della sessione corrente in memoria. 5. Il sistema ricarica l'interfaccia mostrando la **Dashboard specifica** per il nuovo ruolo
Alternativa/e	1b **Utente mono-ruolo**: Se l'utente possiede un solo ruolo, il sistema nasconde l'opzione "Cambia Ruolo"
Post-condizioni	La sessione rimane attiva e l'interfaccia riflette i privilegi e le viste del nuovo ruolo selezionato

Nome	**Visualizza notifiche**
Scopo	Permettere all'utente di consultare gli avvisi riguardanti il cambio di stato delle attività o nuove richieste di approvazione.
Attore/i	Utente Autenticato
Pre-condizioni	L'utente è autenticato
Trigger	L'utente clicca sull'icona delle notifiche
Descrizione sequenza eventi	1. Il sistema mostra un pannello (o elenco) con le ultime notifiche ricevute, ordinate dalla più recente. 2. Il sistema evidenzia visivamente le notifiche "Non lette". 3. Per ogni notifica, il sistema mostra: Titolo Attività, Tipo Evento (es. "Approvata", "Rifiutata", "Nuova richiesta") e Data. 4. L'utente clicca su una specifica notifica. 5. Il sistema marca la notifica come "Letta". 6. Il sistema reindirizza l'utente direttamente alla pagina di dettaglio dell'attività associata.
Alternativa/e	1b. **Nessuna notifica**: Il sistema mostra il messaggio "Nessuna notifica presente".

	6b **Attività cancellata**: Se l'attività a cui si riferisce la notifica è stata eliminata nel frattempo, il sistema mostra un avviso: "L'attività associata non è più disponibile 6c. **notifica di rifiuto**: Se la notifica riguardava un Rifiuto L'utente visualizza nella pagina dell'attività il campo "Motivazione/Feedback
Post-condizioni	La notifica selezionata risulta letta e l'utente si trova nella schermata di dettaglio dell'attività pertinente.

Nome	**Visualizza elenco attività**
Scopo	Permettere all'utente di visualizzare la lista delle attività di propria competenza e verificarne lo stato di avanzamento
Attore/i	Utente Autenticato
Pre-condizioni	L'utente è autenticato
Trigger	L'utente accede al sistema o clicca su "Attività"
Descrizione sequenza eventi	1. Il sistema esegue la query al database applicando i filtri di visibilità in base al proprio ruolo attivo 2. Il sistema mostra la lista tabellare con i dati sintetici: Titolo, Promotore, Stato, Data inizio, Data fine 3. L'utente utilizza la barra di ricerca avanzata per filtrare ulteriormente per periodo o tipologia 4. L'utente seleziona una riga per accedere al dettaglio.
Alternativa/e	2b. **Lista Vuota**: Se non sono presenti attività il sistema mostra “Nessuna attività presente”
Post-condizioni	L'utente visualizza l'elenco dei dati filtrato secondo i permessi del proprio ruolo.

Nome	**Crea attività**
Scopo	Permettere al docente di inserire una nuova attività di Terza Missione nel sistema per avviare il processo di rendicontazione
Attore/i	Docente
Pre-condizioni	Il docente deve essere autenticato nel sistema
Trigger	Il docente seleziona l’opzione “Nuova Attività” dalla dashboard o dal menu
Descrizione sequenza eventi	1. Il sistema mostra il form di inserimento per una nuova attività 2. Il docente inserisce i dati obbligatori: titolo, descrizione, data inizio, data fine… 3. Il docente seleziona la tipologia dal menu a tendina 4. Il docente carica eventuali allegati. Il sistema verifica immediatamente formato (PDF/JPG), dimensione (<5MB) e assenza di virus 5. Il docente preme il pulsante salva bozza 6. Il sistema verifica la correttezza formale dei dati inseriti 7. Il sistema salva l’attività nel database assegnandole id univoco 8. Il sistema conferma l’avvenuto salvataggio mostrandola nella lista delle bozze
Alternativa/e	3b. **Gestione public engagement**:

	se al passo 3 il docente seleziona divulgazione e public engagement il sistema rende visibile e obbligatorio il campo Numero stimato di partecipanti. Il docente deve inserire un valore per poi continuare la sequenza standard 6b. **Errore validazione:** Se mancano dati obbligatori (o se Data Fine < Data Inizio), il sistema evidenzia i campi errati e non permette il salvataggio.
Post-condizioni	Una nuova istanza di attività è creata nel sistema con stato **BOZZA**

Nome	**Modifica Attività**
Scopo	Permettere al docente di aggiornare, correggere o completare i dati di un'attività precedentemente salvata.
Attore/i	Docente
Pre-condizioni	1. Il docente è autenticato 2. Esiste un'attività creata dal docente che si trova nello stato BOZZA
Trigger	Il Docente, dopo aver consultato le proprie attività seleziona una specifica attività in stato BOZZA e clicca su "Modifica"
Descrizione sequenza eventi	1. Il sistema recupera i dati attuali dell'attività dal database. 2. Il sistema mostra il form precompilato con i dati esistenti. 3. Il docente modifica uno o più campi (es. aggiunge una descrizione più dettagliata o carica un nuovo allegato. Se aggiunto un allegato vengono effettuati gli stessi controlli sugli allegati previsti dal caso d'uso crea attività). 4. Il docente preme il pulsante "Salva Modifiche". 5. Il sistema valida nuovamente i dati (es. controlla che i campi obbligatori non siano stati svuotati). 6. Il sistema aggiorna il database 7. Il sistema conferma l'aggiornamento e riporta l'utente alla lista delle attività
Alternativa/e	2b. **Attività preveniente da rifiuto**: il sistema mostra in evidenza il box "Ultimo Feedback/Motivazione" 5b. **Errore validazione:** Se mancano dati obbligatori (o se Data Fine < Data Inizio), il sistema evidenzia i campi errati e non permette il salvataggio.
Post-condizioni	I dati dell'attività sono aggiornati nel sistema. Lo stato rimane **BOZZA**

Nome	**Elimina attività**
Scopo	Permettere al docente di eliminare definitivamente dal sistema una propria attività in stato di BOZZA
Attore/i	Docente
Pre-condizioni	1. Il docente è autenticato nel sistema. 2. L'attività selezionata appartiene al docente e si trova nello stato di BOZZA
Trigger	Il docente seleziona l'opzione "Elimina" nella schermata di modifica dell'attività
Descrizione sequenza eventi	1. Il docente preme il pulsante per eliminare l'attività 2. Il sistema procede all'eliminazione di tutti gli allegati sul Server

	3. Il sistema procede alla rimozione fisica del record dal database e di tutte le entità collegate
Post-condizioni	L'attività è stata eliminata

Nome	**Invia Attività**
Scopo	Finalizzare l'inserimento dell'attività e sottometterla formalmente all'approvazione del Responsabile di Struttura.
Attore/i	Docente
Pre-condizioni	1. Il docente è autenticato nel sistema. 2. Il docente ha selezionato un'attività precedentemente salvata e attualmente in stato di BOZZA. 3. I campi obbligatori minimi dell'attività sono già stati compilati e consolidati a sistema.
Trigger	Il docente preme il pulsante "Invia" dalla schermata di dettaglio dell'attività
Descrizione sequenza eventi	1. Il sistema cambia lo stato dell'attività da BOZZA a INVIATA. 2. Il sistema blocca l'attività in modalità Sola Lettura (Read-Only). 3. Il sistema invia una notifica e una mail al Responsabile di Struttura. 4. Il sistema conferma l'operazione al docente
Alternativa/e	2b: **Annullamento utente**: L'utente seleziona "Annulla" alla richiesta di conferma. Il caso d'uso termina senza cambiare stato e il controllo ritorna al caso d'uso base (l'attività resta in Bozza e modificabile).
Post-condizioni	L'attività è in stato INVIATA e non è più modificabile dal docente

Nome	**Valuta attività**
Scopo	Permettere al Responsabile di Struttura di esaminare i contenuti di un'attività e approvarla o rifiutarla.
Attore/i	Responsabile di Struttura
Pre-condizioni	Esiste almeno un'attività in stato "INVIATA" e appartiene ad un docente della stessa struttura del Responsabile. Il responsabile non è l'autore dell'attività
Trigger	Il Responsabile, dopo aver consultato l'elenco delle attività, seleziona una specifica attività in stato "INVIATA".
Descrizione sequenza eventi	1. Il sistema mostra la scheda di dettaglio dell'attività in modalità sola lettura. 2. Il Responsabile esamina i dati inseriti ed eventuali allegati. 3. Il Responsabile preme il pulsante "Approva" 4. Il sistema chiede conferma dell'operazione 5. Il Responsabile conferma. 6. Il sistema aggiorna lo stato dell'attività da INVIATA ad APPROVATA.

	7. Il sistema informa con una mail e una notifica al docente (autore)
Alternativa/e	3b: **Rifiuto con motivazione**: Il Responsabile riscontra problemi e preme "Rifiuta". Il sistema costringe ad inserire la Motivazione del rifiuto. Il Responsabile inserisce il testo e conferma. Il sistema riporta lo stato dell'attività a BOZZA e salva la motivazione (visibile al docente per le correzioni).
Post-condizioni	L'attività passa allo stato APPROVATA (successo) oppure torna allo stato BOZZA (rifiuto).

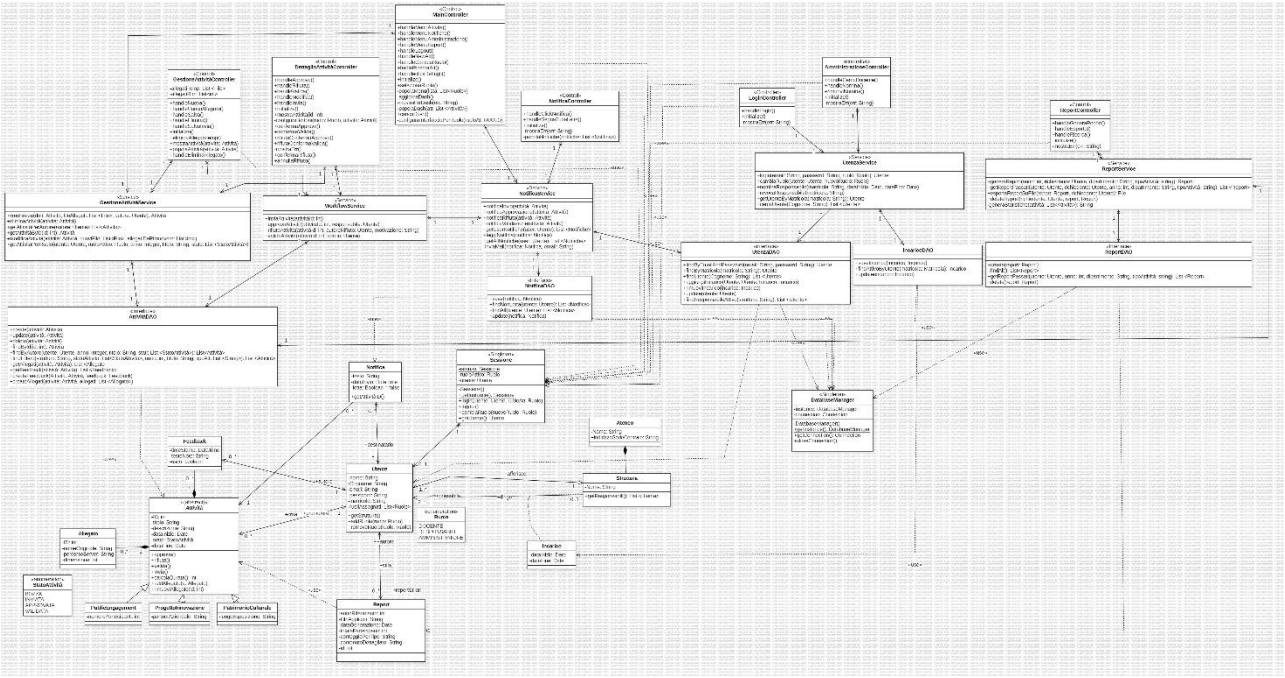
Nome	**Valida Attività**
Scopo	Verificare la conformità formale dell'attività approvata per includerla nel rendiconto istituzionale annuale.
Attore/i	Amministrazione
Pre-condizioni	1. Esiste un'attività in stato APPROVATA.
Trigger	L'Amministratore, dopo aver consultato l'elenco delle attività, seleziona una specifica attività in stato "APPROVATA".
Descrizione sequenza eventi	1. Il sistema mostra la scheda di dettaglio dell'attività in modalità sola lettura. 2. L'amministratore la controlla 3. L'amministratore preme il pulsante "Valida". 4. Il sistema chiede conferma. 5. L'amministratore conferma. 6. Il sistema aggiorna lo stato dell'attività da APPROVATA a VALIDATA. 7. Il sistema invia una notifica e una mail di successo al Docente
Alternativa/e	3b: **Richiesta Modifiche**: L'amministratore preme "Rifiuta". Il sistema rende obbligatorio il campo "Motivazione". L'amministratore inserisce il testo e conferma. Il sistema: Riporta lo stato a BOZZA, Salva Motivazione, Autore e Timestamp in un feedback, Invia notifica e mail al Docente con l'estratto del feedback
Post-condizioni	L'attività passa allo stato VALIDATA (pronta per il report) oppure torna allo stato BOZZA (da correggere).

Nome	**Gestione Incarichi Responsabile**
Scopo	Gestione Incarichi Responsabile (nomina/revoca)
Attore/i	Amministrazione
Pre-condizioni	1. L'Amministratore è autenticato 2. Il docente è già censito nel sistema anagrafico di Ateneo
Trigger	L'Amministratore seleziona la voce "Gestione Nomine" dalla dashboard.
Descrizione sequenza eventi	1. L'Amministratore accede alla sezione di gestione degli incarichi. 2. L'Amministratore inserisce la matricola del Docente da nominare, specificando la Data di Inizio e la Data di Fine mandato. 3. L'Amministratore conferma l'operazione. 4. Il sistema verifica l'esistenza della matricola inserita.

	5. Il sistema assegna l'incarico direttivo al Docente, associandolo automaticamente alla Struttura (dipartimento) di sua attuale appartenenza
Alternativa/e	2b: **Revoca Anticipata**: Il sistema verifica che l'utente esista e che abbia un incarico attualmente attivo. Il sistema aggiorna la data di fine mandato nel database. Se la data inserita coincide con la data odierna, il sistema procede alla rimozione immediata dei privilegi di ruolo ("Responsabile") dall'anagrafica dell'utente **5b. Incarico preesistente (Sovrapposizione):** Se il sistema rileva che il Docente indicato possiede già un incarico direttivo attivo, l'operazione viene interrotta e viene mostrato un messaggio di errore ("Il docente è già responsabile"). **4b. Matricola Errata:** Se la matricola inserita non corrisponde a nessun utente a sistema, l'operazione si interrompe con un errore.
Post-condizioni	Il Docente selezionato acquisisce anche il ruolo di Responsabile

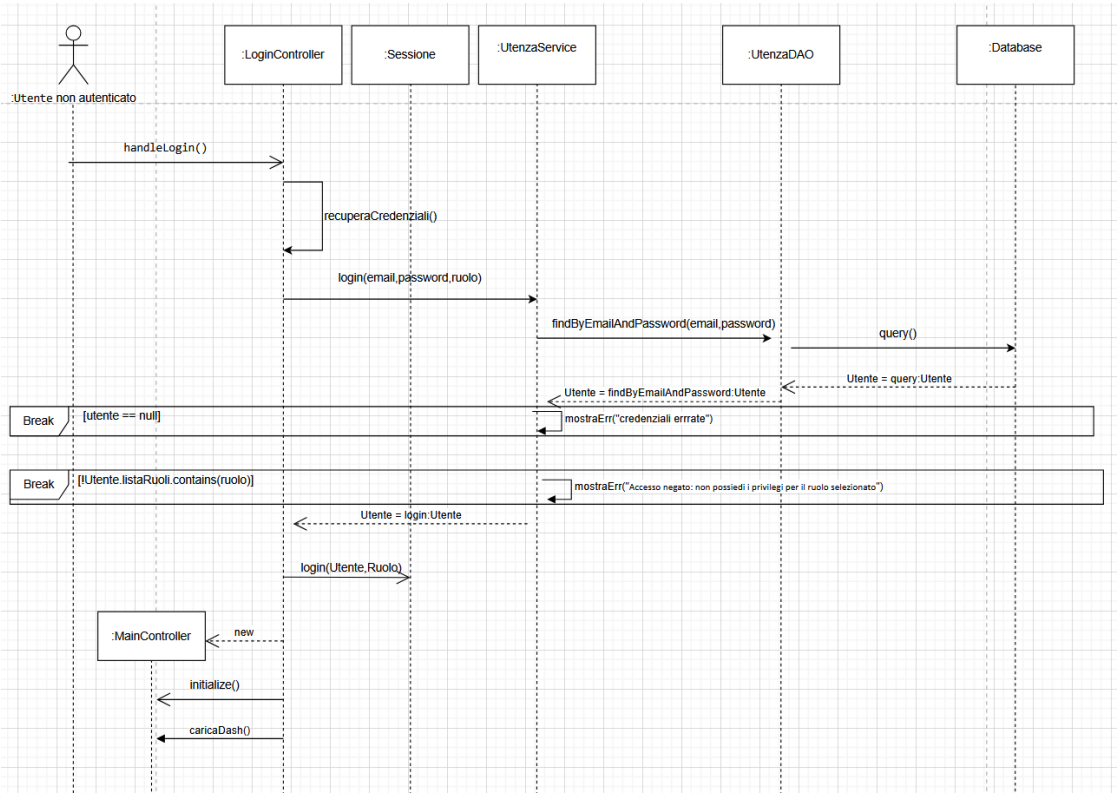
Nome	**Stila Report**
Scopo	Generare un documento ufficiale riepilogativo delle attività di Terza Missione per fini statistici o ministeriali.
Attore/i	Amministrazione
Pre-condizioni	Esistono attività nello stato VALIDATA nel sistema.
Trigger	L'amministratore seleziona la voce "Stila Report" dal menu principale.
Descrizione sequenza eventi	1. Il sistema mostra una maschera per la selezione dei criteri di filtro (es. Anno Accademico, Dipartimento, Tipologia Attività). 2. L'amministratore imposta i filtri desiderati e preme "Genera Report". 3. Il sistema recupera tutte le attività VALIDATE che soddisfano i criteri e ne aggregherà i dati (es. somma dei partecipanti, conteggio attività per tipo). 4. Il sistema mostra i dati
Alternativa/e	4b: Nessun dato trovato: Se nessuna attività corrisponde ai filtri selezionati, il sistema mostra un avviso: "Nessuna attività trovata per i criteri selezionati
Post-condizioni	Il report è stato generato

2.3 Diagramma delle classi



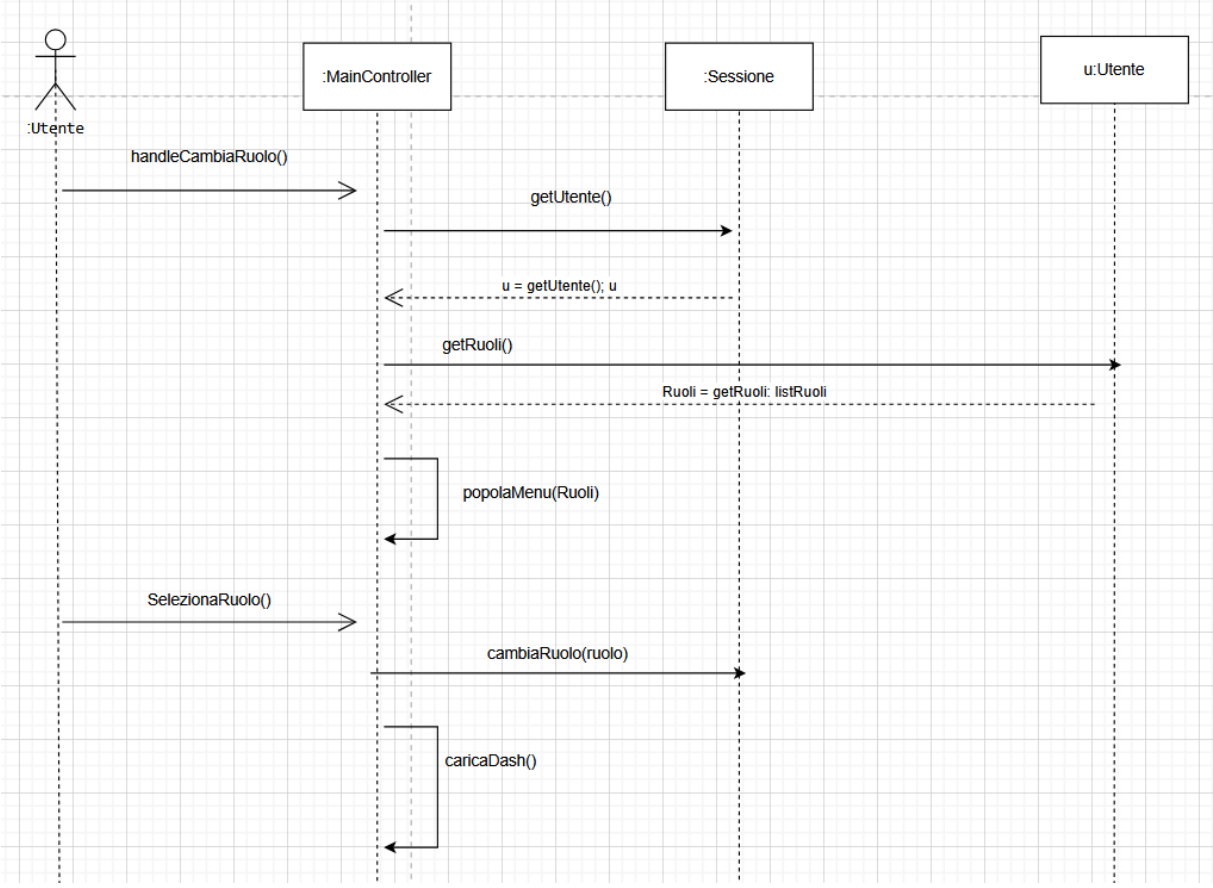
2.4 Diagrammi di sequenza

Login:
Questo diagramma mostra il caso d'uso login, dove l'utente inserisce le proprie credenziali e il ruolo nel quale vuole accedere, il sistema verifica contattando il database per controllare se le credenziali sono corrette



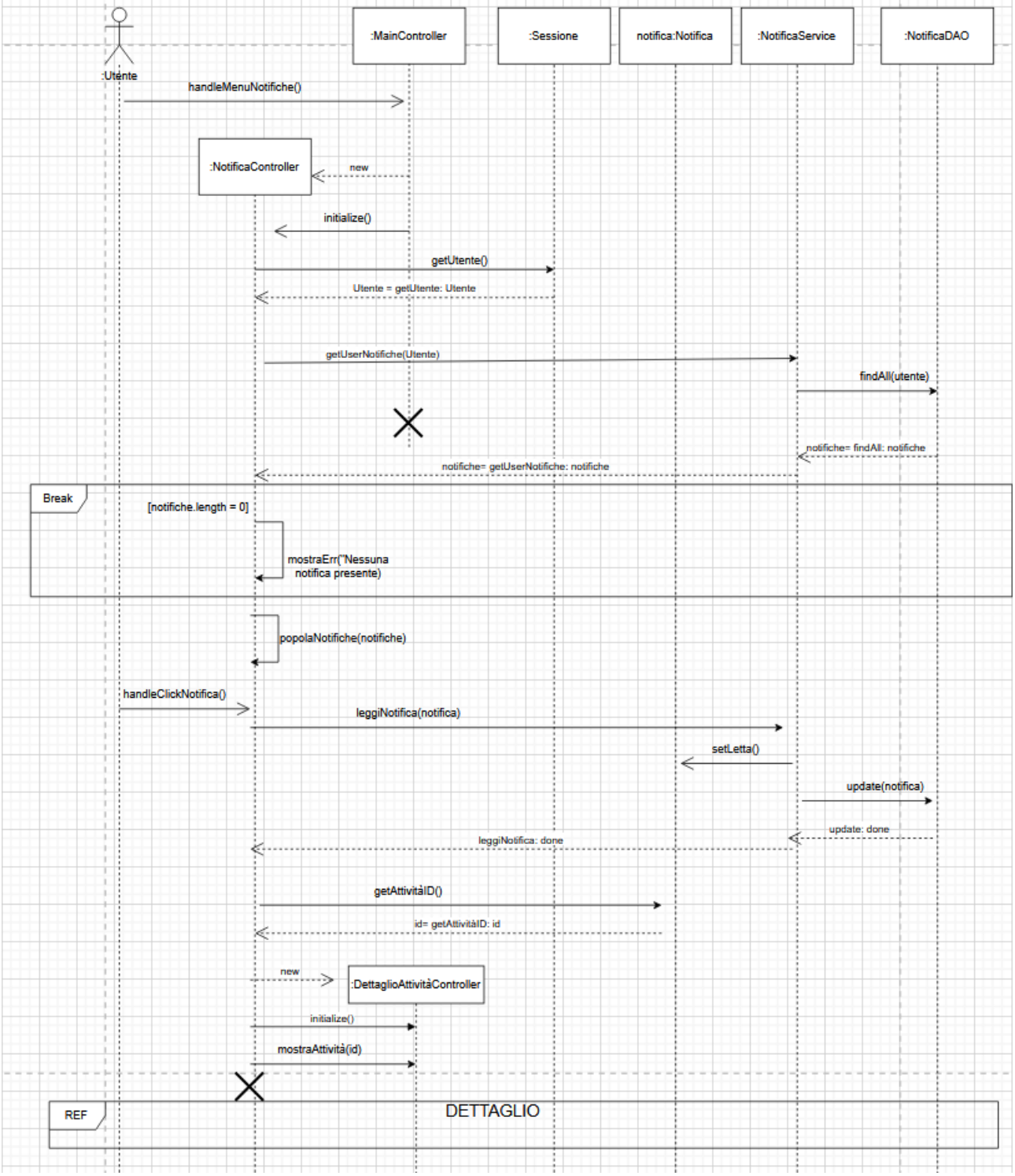
Cambia Ruolo:

Questo diagramma descrive il processo tramite il quale un utente autenticato seleziona e attiva uno dei suoi altri ruoli. Illustra l'interazione del MainController con la Sessione per recuperare i ruoli disponibili e popolare l'interfaccia. Conclude mostrando il ricaricamento dinamico della dashboard, che adatterà istantaneamente le funzionalità visibili in base ai nuovi privilegi.

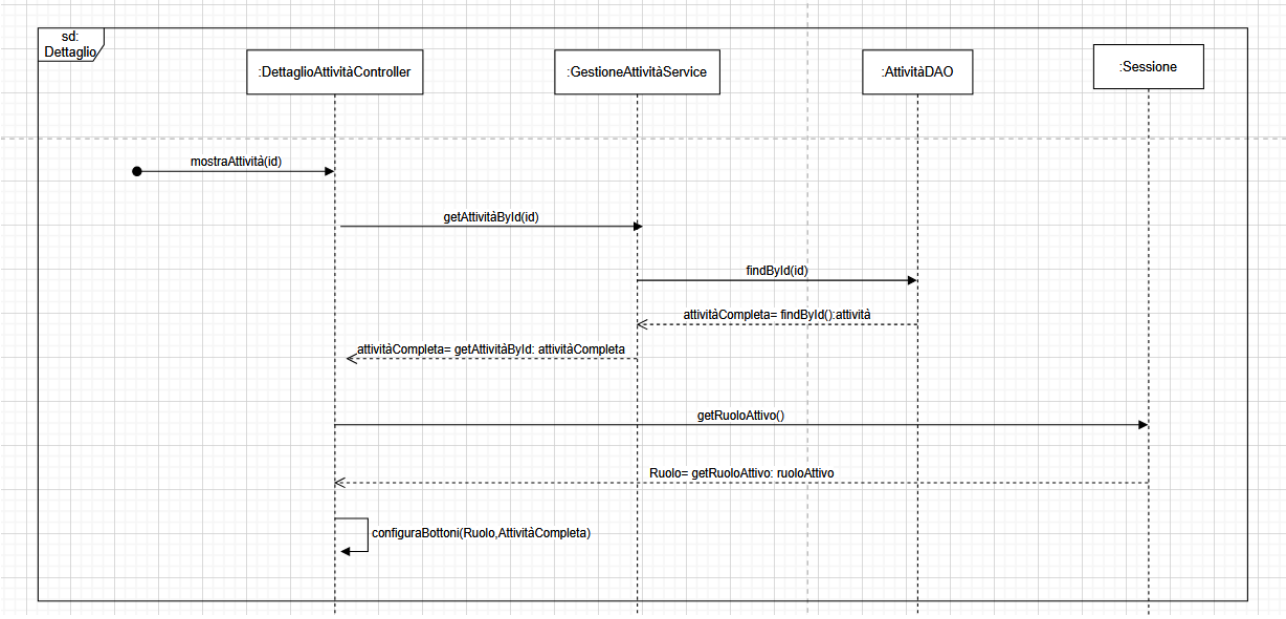


Visualizza Notifiche:

Questo diagramma mostra la consultazione e interazione con le notifiche dell'utente. Illustra il recupero dello storico dal database tramite il Service, il popolamento dell'interfaccia e la gestione del click su uno specifico avviso. L'interazione innesca l'aggiornamento persistente dello stato della notifica (marcata come "letta") e il reindirizzamento automatico al dettaglio dell'attività correlata.

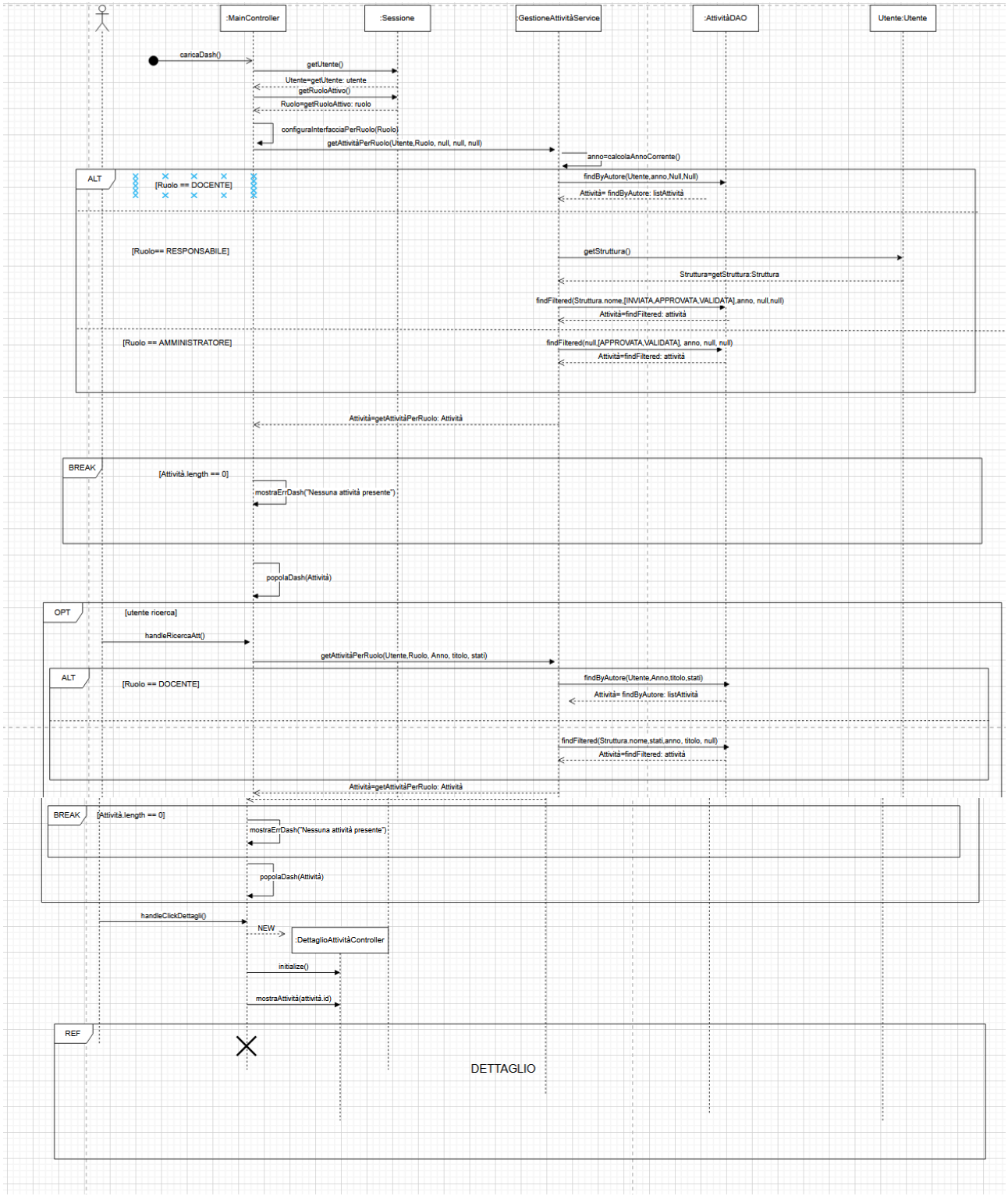


Dettaglio:
Questo frammento di diagramma illustra il caricamento delle informazioni complete di una singola attività a partire dal suo identificativo (ID). Mostra l'interazione del Controller con i layer sottostanti per recuperare i dati aggiornati dal database e la successiva interrogazione della Sessione; quest'ultimo passaggio permette al sistema di configurare dinamicamente l'interfaccia e i permessi dei bottoni in base al ruolo attivo dell'utente.



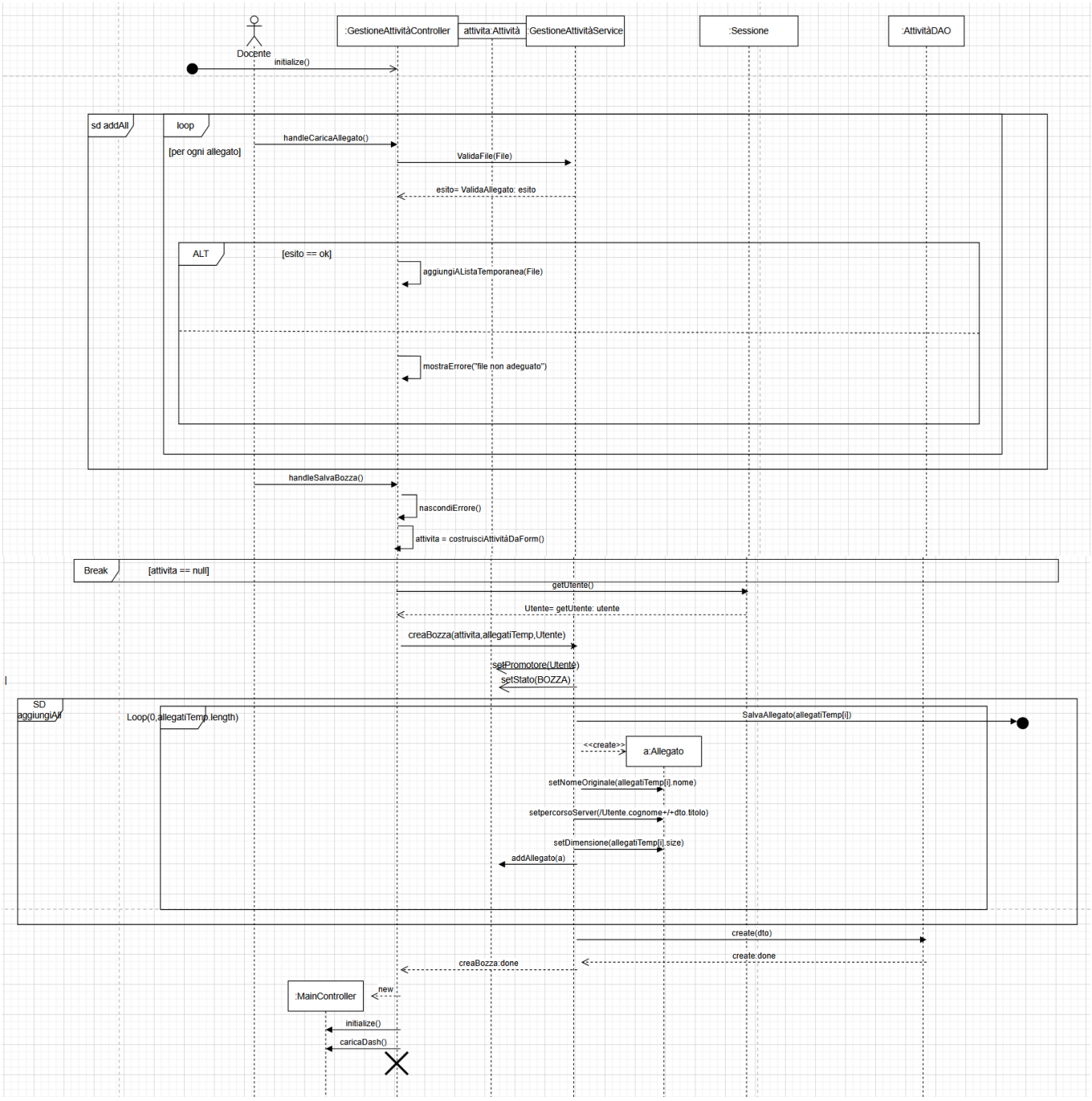
Visualizza elenco attività:

Questo diagramma descrive il processo di inizializzazione e popolamento della dashboard principale. Illustra come il Controller interroghi la Sessione per identificare il ruolo dell'utente, delegando al Service e al DAO l'estrazione mirata dei record in base ai privilegi. Evidenzia inoltre il flusso opzionale per la ricerca filtrata e l'accesso ai dettagli della singola entità tramite passaggio del solo ID

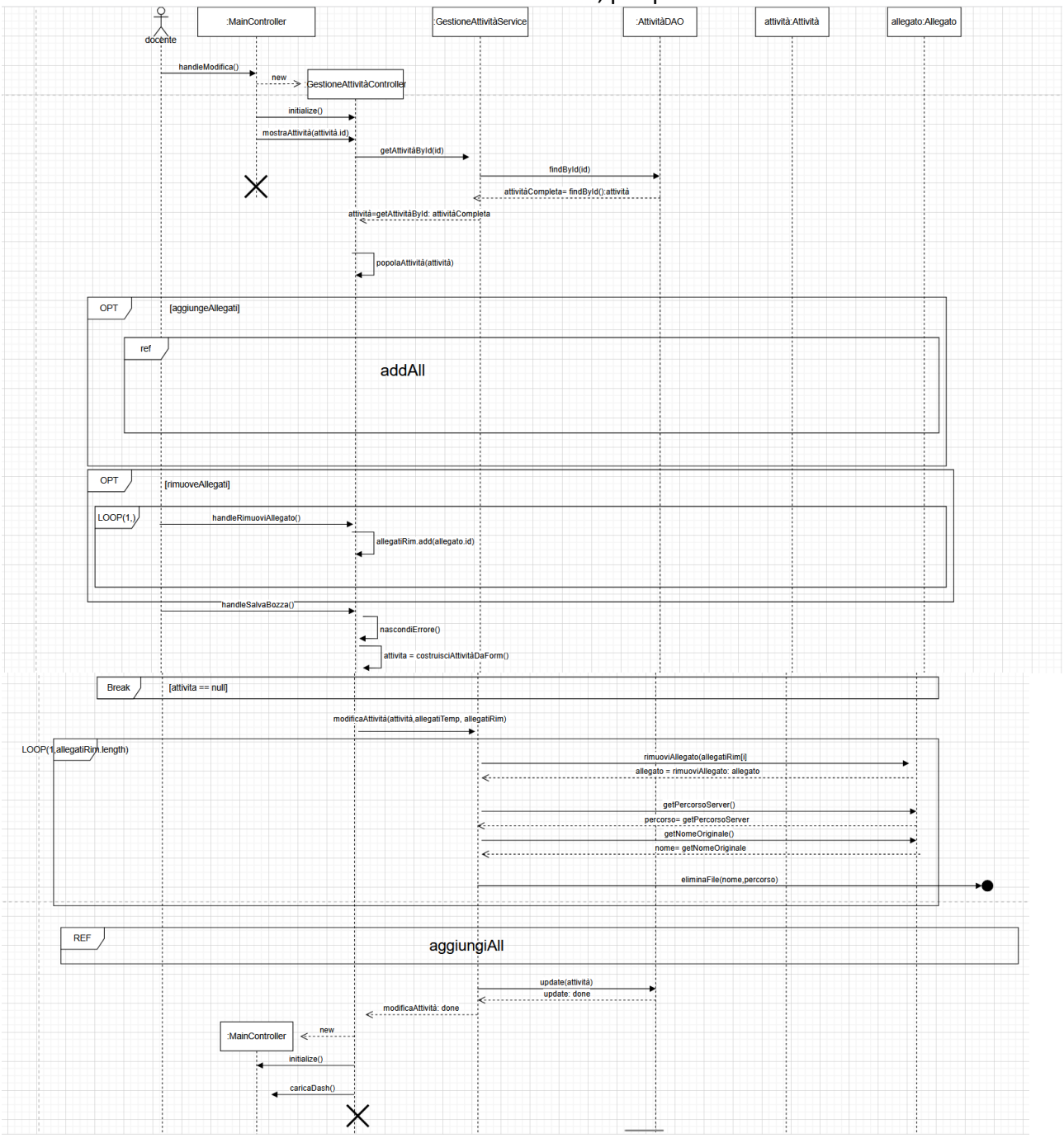


Crea Bozza:

Questo diagramma descrive la creazione di una nuova attività in stato di "Bozza". Illustra la validazione preventiva degli allegati e la costruzione dell'entità nel Controller. Il service orchestra il salvataggio fisico dei file, l'istanziazione degli oggetti correlati e la persistenza finale tramite il DAO, associando dinamicamente l'autore recuperato dalla Sessione.

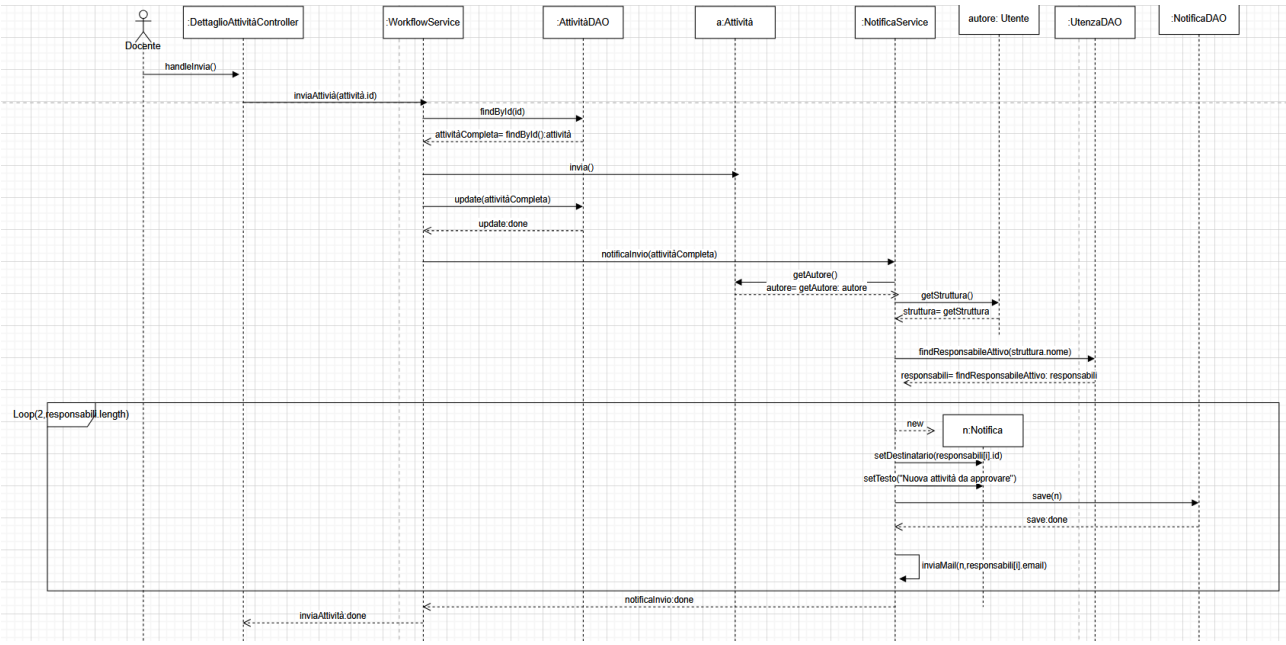


Modifica Bozza:
Questo diagramma descrive il flusso di aggiornamento di un'attività in stato di "Bozza”. Mostra il caricamento dei dati dell’attività. Delegando poi al Service la complessa orchestrazione del salvataggio. Quest'ultimo si occupa di eliminare dal server i file scartati, salvare i nuovi allegati fisici e salvare l'entità modificata sul database tramite il DAO, per poi ricaricare la dashboard.



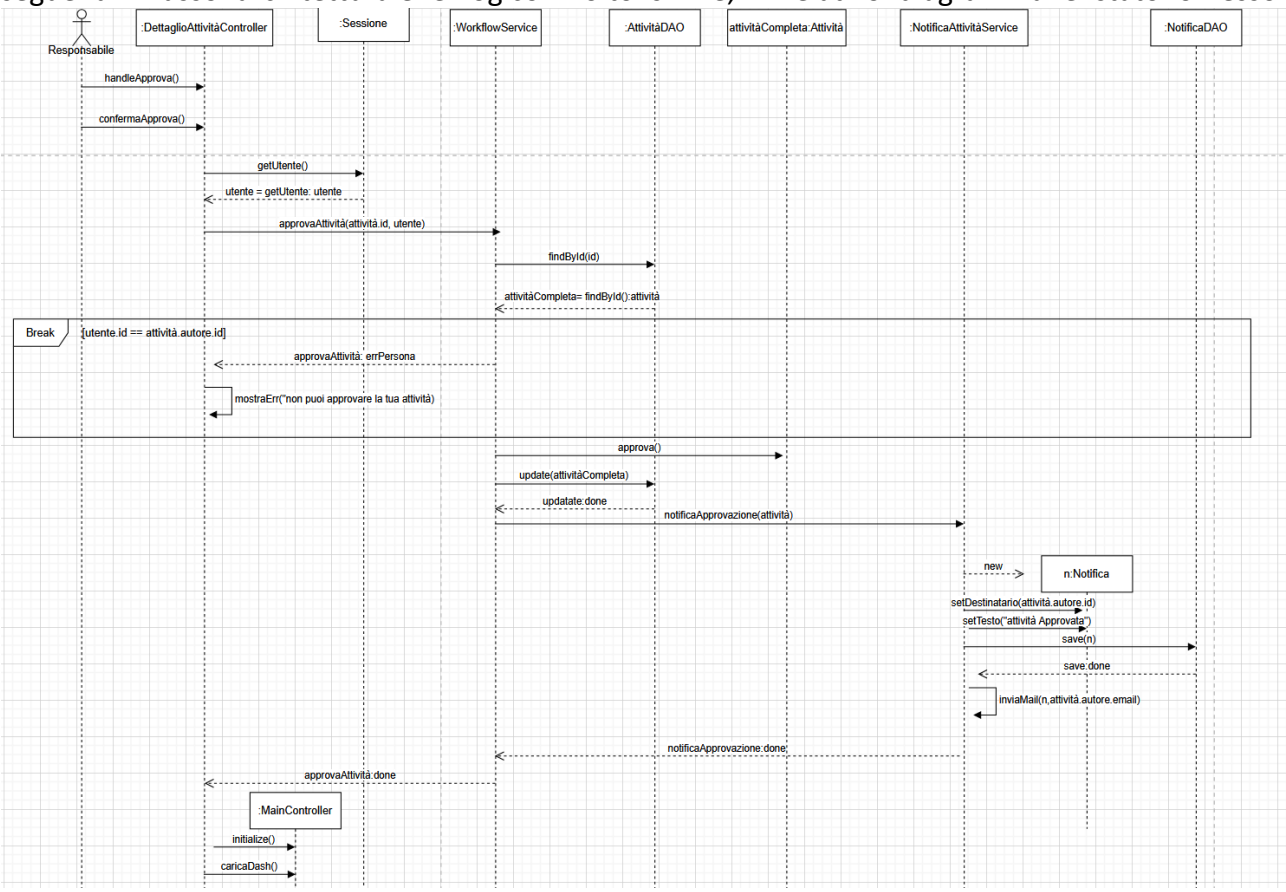
Invia Bozza:
Questo diagramma descrive la sottomissione di un'attività per la fase di revisione. Illustra il cambio di stato logico (in "INVIATA") orchestrato dal WorkflowService seguito dalla delega al NotificaService. Quest'ultimo identifica dinamicamente i Responsabili di dipartimento competenti

in base alla struttura dell'autore, generando le notifiche persistenti (tramite DAO) e inoltrando le relative email.

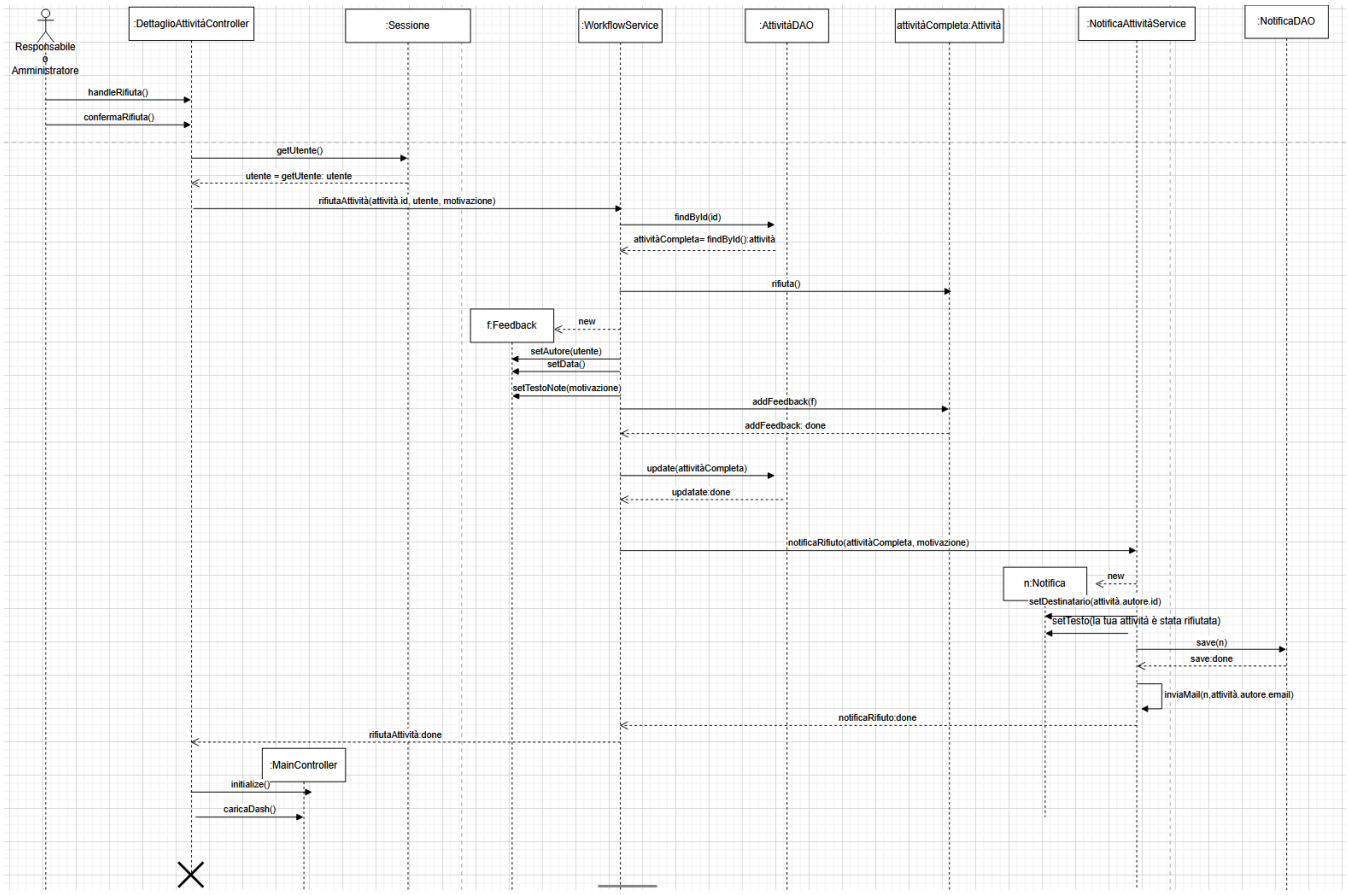


Approva:

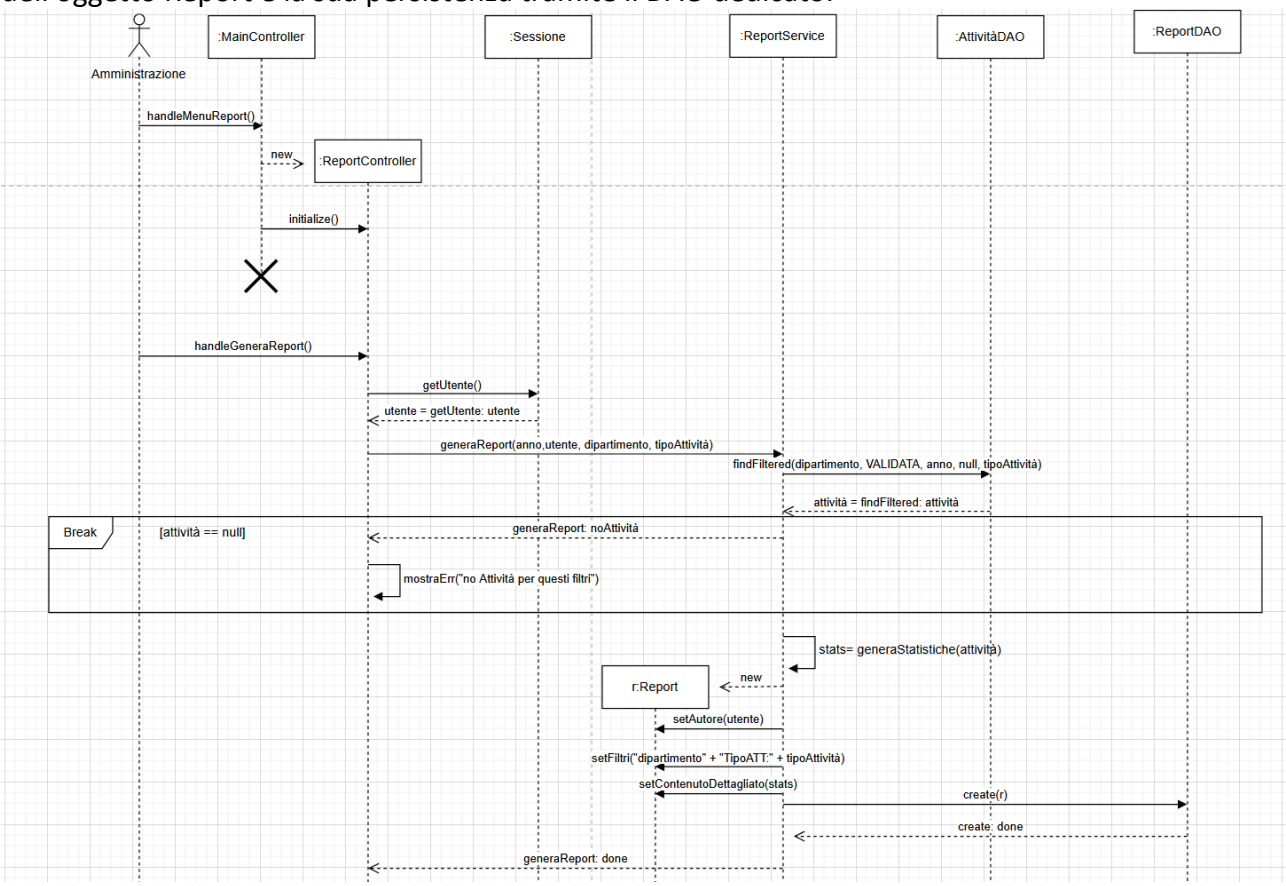
Questo diagramma descrive il processo di approvazione di un'attività sottomessa. Evidenzia il controllo della regola di business da parte del WorkflowService (impossibilità di auto-approvazione tramite il blocco Break), l'aggiornamento dello stato sul database e la generazione automatica della notifica di esito per l'autore. Poiché il successivo caso d'uso "Valida" (eseguito dall'Amministratore) segue un flusso architetturale e logico molto simile, il relativo diagramma è stato omesso.



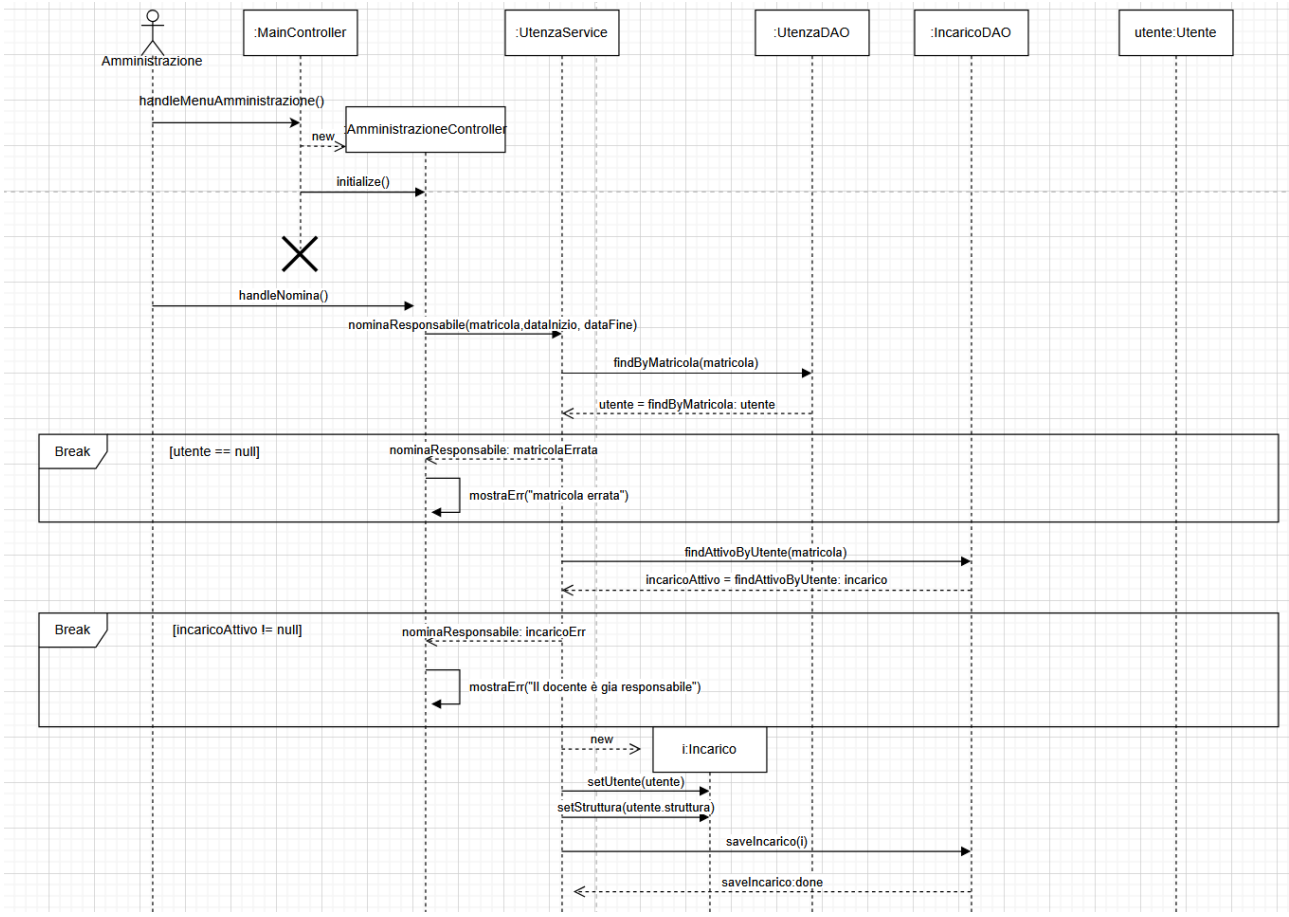
Rifiuta:
Questo diagramma descrive il processo di rifiuto di un'attività sottomessa, un caso d'uso condiviso sia dal Responsabile che dall'Amministratore. Illustra come il WorkflowService gestisca la retrocessione logica dello stato dell'entità (ritorno a "BOZZA") e la contestuale creazione di un oggetto "Feedback" per tracciarne le motivazioni. Il flusso si chiude con il salvataggio sul database, l'inoltro della notifica all'autore e il ritorno alla dashboard.



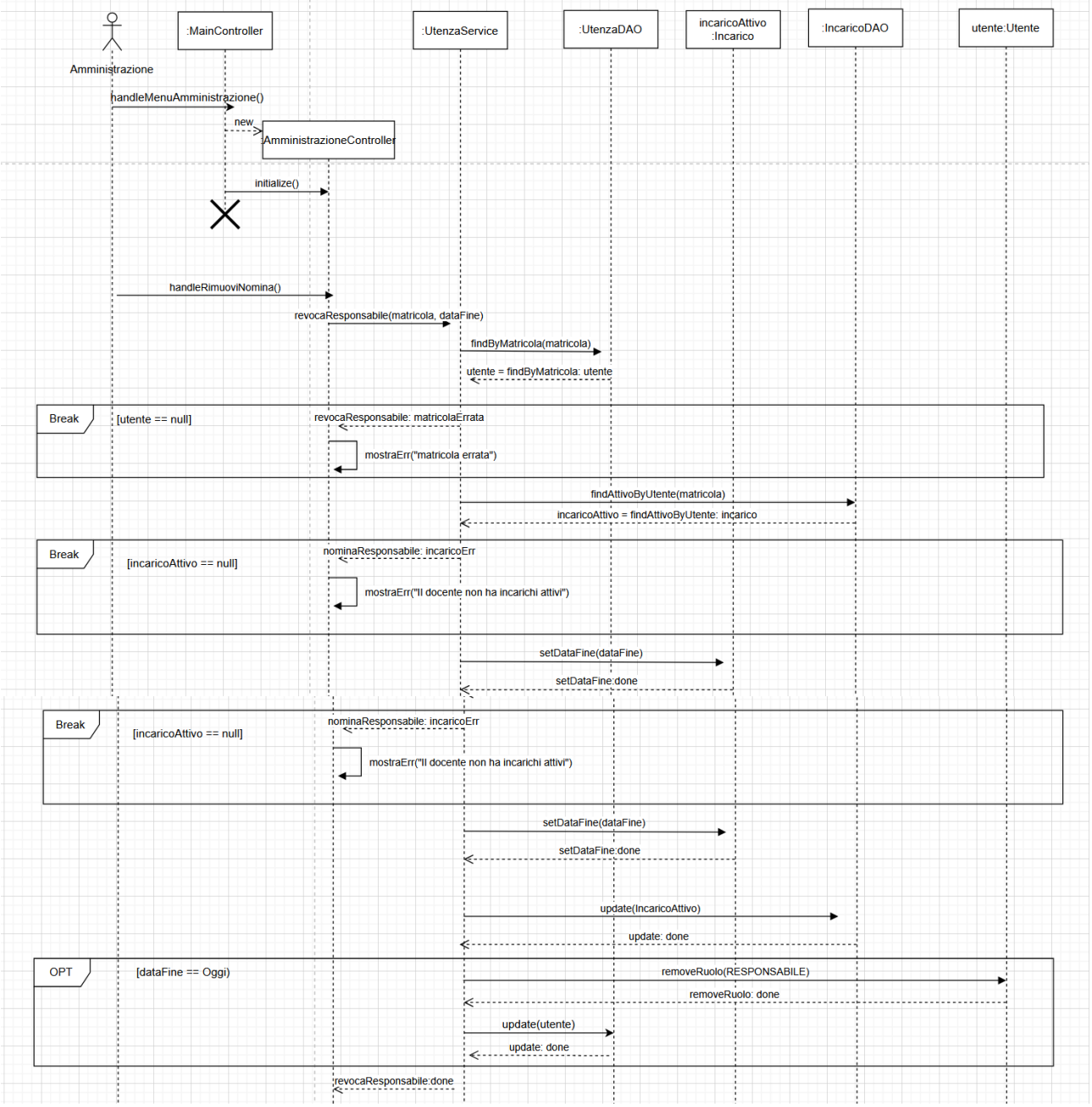
Stila Report:
Questo diagramma descrive il processo di generazione di un report statistico da parte dell'Amministratore. Illustra l'interazione del Controller con il ReportService per estrarre dal database le sole attività validate in base ai filtri selezionati, gestendo tramite un blocco Break l'eventualità di assenza di dati. Il flusso si conclude mostrando il calcolo delle statistiche, la creazione dell'oggetto Report e la sua persistenza tramite il DAO dedicato.



Nomina responsabile:
Questo diagramma descrive il processo con cui l'Amministratore assegna il ruolo di Responsabile a un docente. il flusso verifica sequenzialmente l'esistenza della matricola inserita e l'assenza di incarichi direttivi già attivi per quell'utente, interrompendosi in caso di anomalie. Solo superati questi controlli di integrità, il sistema procede all'istanziazione e alla persistenza del nuovo Incarico.



Revoca responsabile:
Questo diagramma descrive il processo di revoca di un incarico direttivo da parte dell'Amministratore. Mostra i controlli preventivi del Service sull'esistenza dell'utente e di un incarico attivo, seguiti dall'aggiornamento della data di termine sul database e l'aggiornamento dei permessi, rimuovendo attivamente il ruolo di "Responsabile" dall'utente.

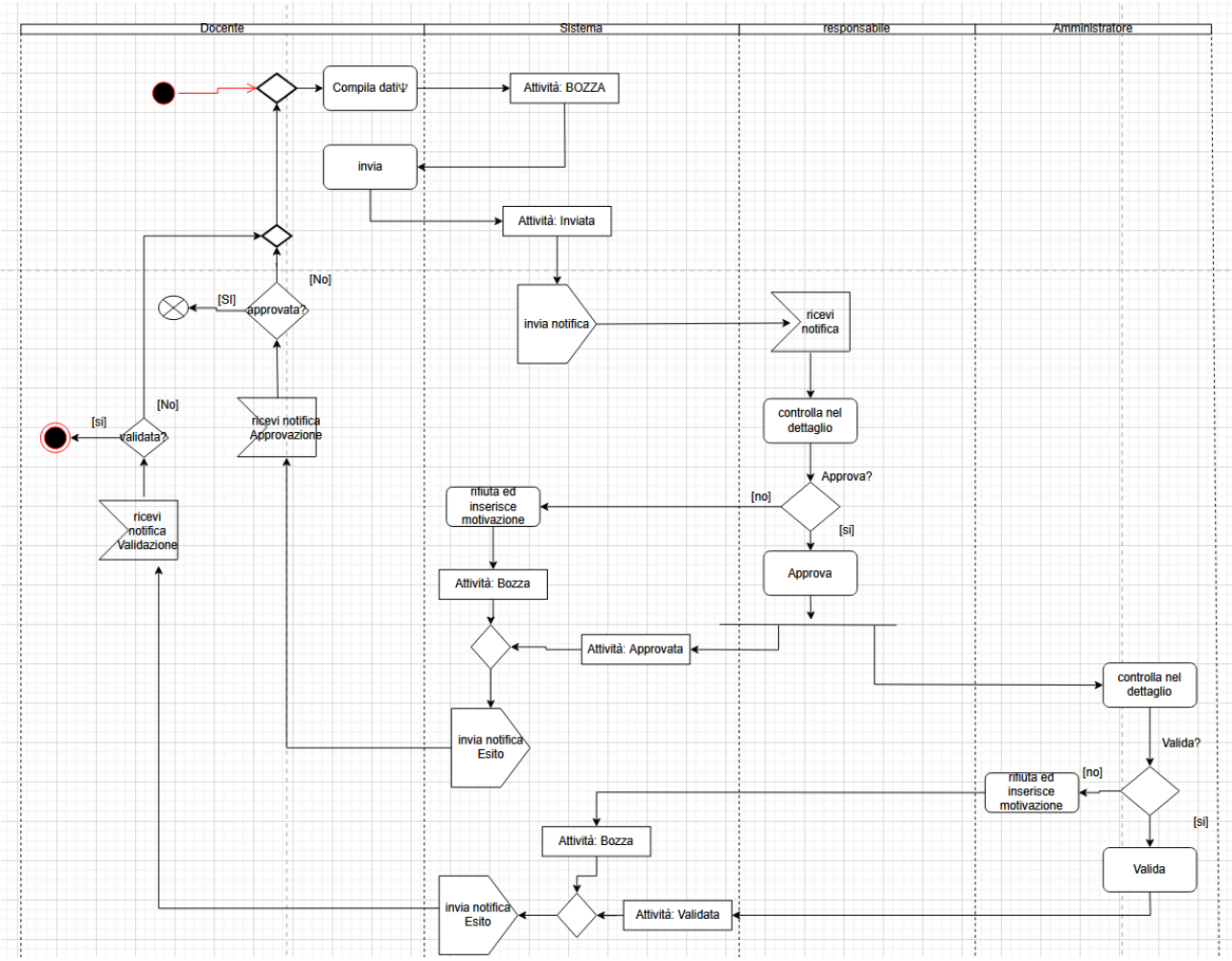


2.5 Diagrammi delle attività

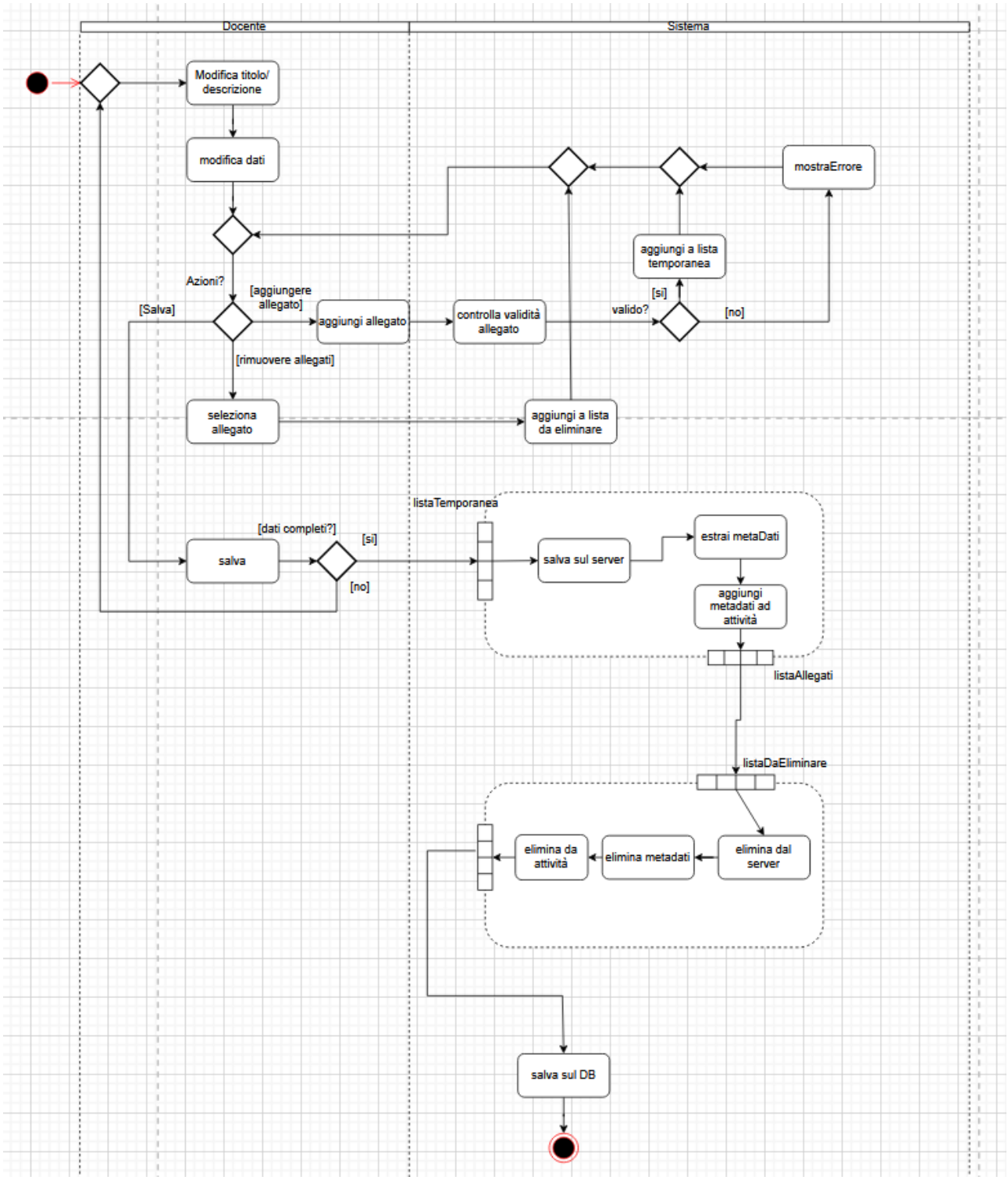
Nei diagrammi di attività presentati, la corsia (swimlane) denominata "Sistema" rappresenta un'astrazione di alto livello dell'infrastruttura dell'applicativo. Per mantenere i diagrammi focalizzati sul workflow logico dell'utente senza appesantirli con dettagli architetturali, il "Sistema" ingloba le responsabilità combinate di tutti i layer applicativi: il Controller (gestione della UI), il Service (logica di business) e il DAO (interazione con il database).

WorkFlow attività:
Questo diagramma illustra il workflow completo relativo al ciclo di vita di una singola attività. Descrive le interazioni tra il Docente, responsabile della compilazione e dell'invio dell'attività, e le

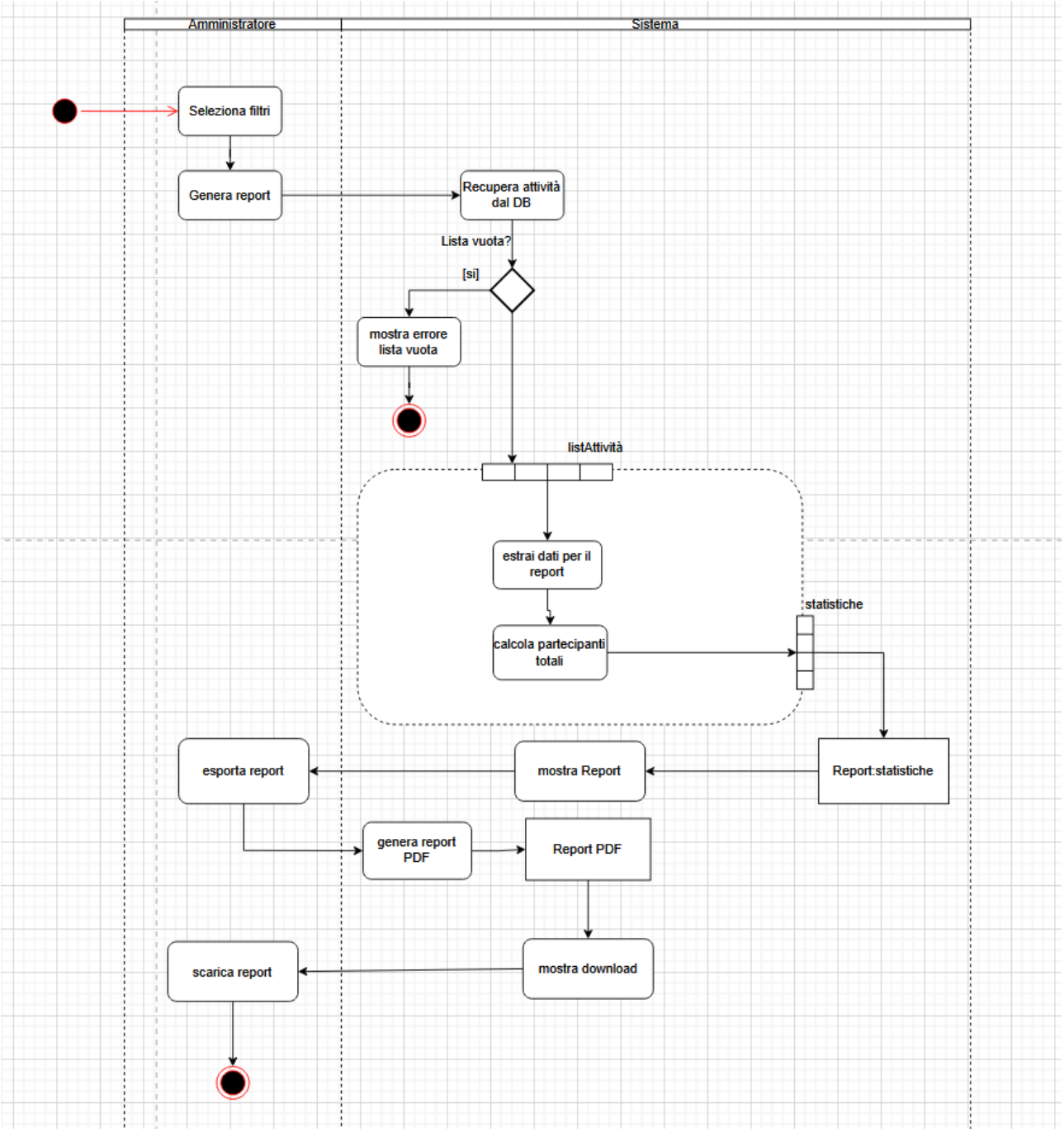
figure di Responsabile e Amministratore, incaricate delle fasi di revisione, approvazione e validazione. La corsia dedicata al Sistema mostra l'orchestrazione automatica del processo, inclusi i cambi di stato dell'attività (da Bozza a Validata) e la gestione dell'invio delle notifiche in base all'esito delle revisioni.



Compila Dati:
Questo diagramma rappresenta la call behaviour Compila Dati. Illustra l'interazione tra l'utente e il sistema durante l'inserimento delle informazioni e la gestione degli allegati evidenziando le logiche temporanee di validazione, il salvataggio o la rimozione fisica dei file dal server e il consolidamento finale dei metadati sul database

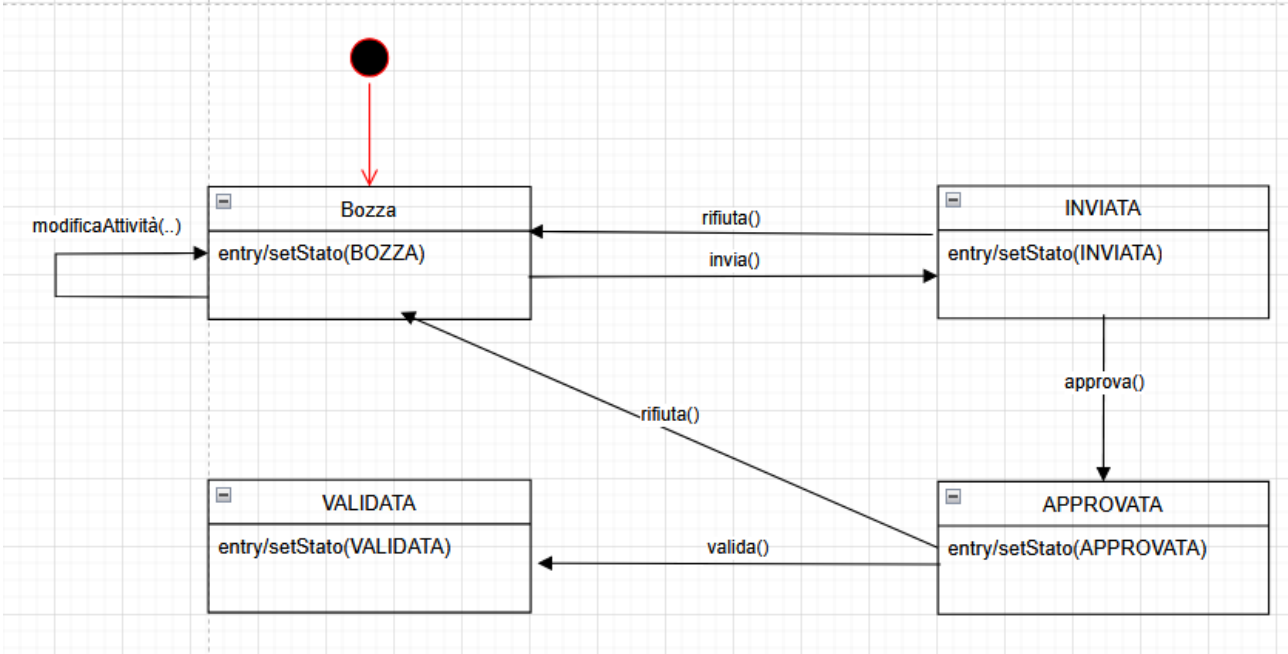


Stila Report:
Questo diagramma illustra il workflow del caso d'uso "Stila Report" riservato all'Amministratore. Mostra l'interazione per la creazione di statistiche basate su filtri: il sistema interroga il database, elabora ciclicamente i dati aggregati (es. partecipanti totali) ed espone i risultati, gestendo infine l'esportazione e il download del documento in formato PDF.

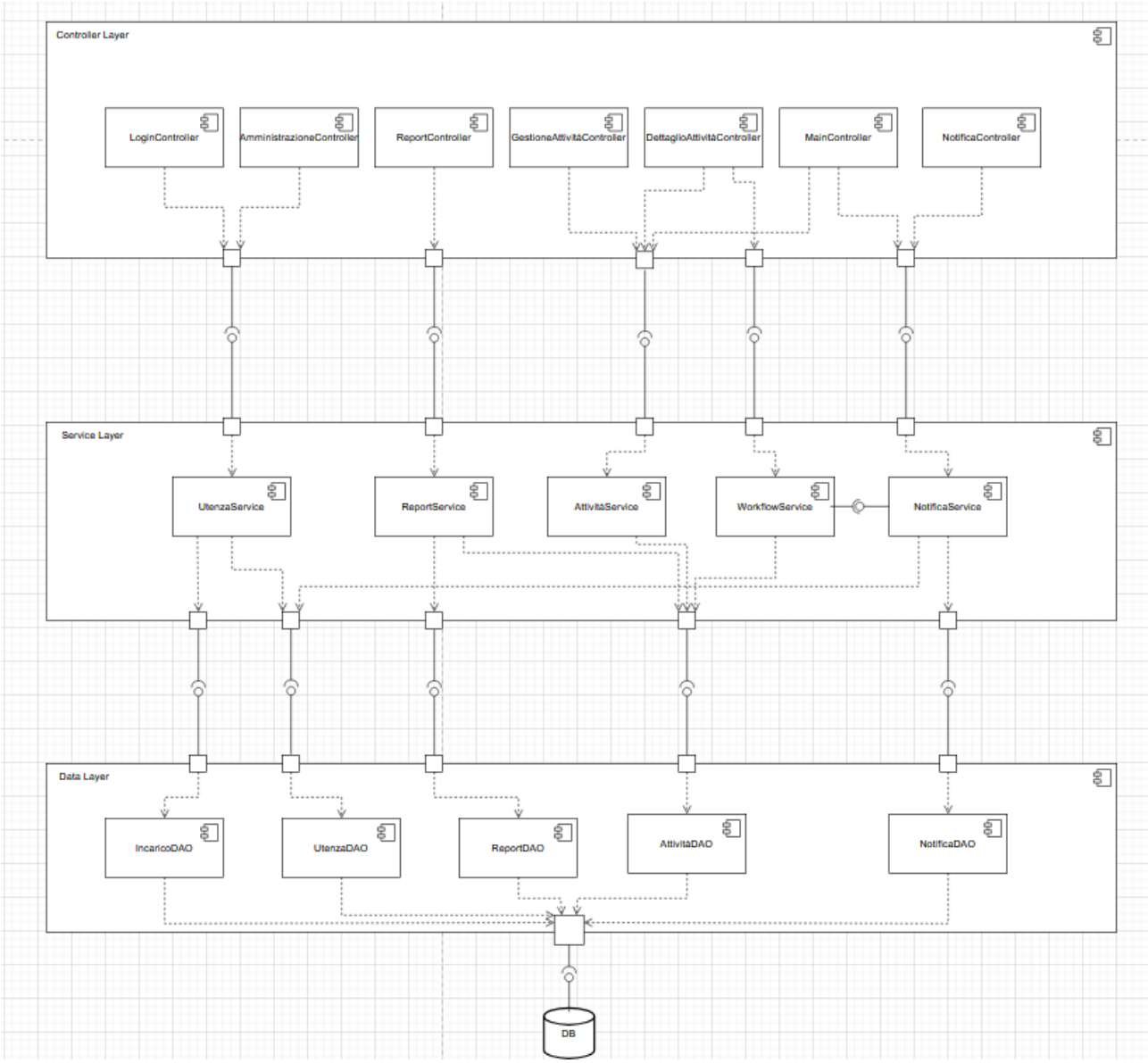


2.6 Macchine di stato

Il diagramma illustra le transizioni di stato principali dell'entità (da Bozza a Validata, incluse le retrocessioni). Si precisa che l'evento modificaAttività è modellato intenzionalmente come una self-transition all'interno dello stato "BOZZA": questo rappresenta un'operazione di dominio significativa che aggiorna i dati dell'entità senza però alterarne lo stato logico, ed è implementata come metodo del gestioneAttivitàService. Le restanti transizioni che comportano un effettivo cambio di stato (invia, approva, valida, rifiuta) corrispondono invece a operazioni pubbliche implementate direttamente all'interno dell'entità "Attività".



2.7 Diagramma dei componenti



3 Implementazione del sistema

Gestione degli Allegati e Semplificazioni Implementative:

Per la gestione dei file allegati alle attività, si è optato per una soluzione di archiviazione su File System locale. Sebbene in un contesto di produzione reale tale compito verrebbe delegato a un server dedicato tramite protocollo FTP, in questa sede si è scelta una **simulazione implementativa** semplificata. I file vengono salvati dinamicamente in una directory radice (/uploads), organizzata in sottocartelle nominate secondo la convenzione Cognome_docente/Titolo_attività/nome_documento.

3.1.1 Discussione dei Design Pattern utilizzati

Model-View-Controller (MVC):

L'obiettivo primario era disaccoppiare l'interfaccia utente (View/FXML) dalla logica applicativa (Controller) e dai dati (Model). Questo permette di modificare la grafica senza rischiare di compromettere la logica di business o la persistenza dei dati. Inoltre, facilita la manutenzione e la divisione del lavoro.

Le classi Controller (es. LoginController) non contengono logica di accesso ai dati, ma agiscono come intermediari: intercettano gli eventi della GUI e delegano le operazioni al livello sottostante

Service Layer:

È stato introdotto il service layer per far sì che i Controller debbano occuparsi esclusivamente della gestione dell'interazione utente (input/output) e della navigazione. Tutta la logica di dominio è delegata ai Service. Garantisce che una regola di business sia implementata in un unico punto, facilitando la manutenzione. Rendendo la logica indipendente dall'interfaccia grafica (JavaFX), il sistema risulta predisposto per eventuali future evoluzioni

Nel diagramma delle classi questo pattern ha comportato: l'introduzione di classi Service intermedie, l'eliminazione di dipendenze dirette tra Controller e DAO, una chiara direzionalità delle dipendenze (Controller → Service → DAO). Questo migliora modularità e manutenibilità.

Data Access Object (DAO):

L'applicazione interagisce con un database relazionale tramite JDBC. Scrivere codice SQL direttamente nei Service avrebbe creato un forte accoppiamento con la tecnologia del database. Il pattern DAO astrae l'accesso ai dati, esponendo al Service un'interfaccia semplice e nascondendo i dettagli tecnici delle query SQL.

Nel diagramma delle classi ciò ha comportato: l'introduzione di classi dedicate alla persistenza, la separazione tra logica applicativa (Service) e logica di accesso ai dati, una dipendenza del Service verso il DAO, non verso il database. Questo consente di modificare la tecnologia di persistenza senza impattare il dominio o la UI.

Singleton:

La connessione al database è una risorsa "pesante" e condivisa. È necessario garantire che esista una e una sola istanza del gestore della connessione per tutta la durata dell'applicazione, centralizzando la configurazione (URL, credenziali) ed evitando conflitti o spreco di risorse.

Sessione Utente le informazioni relative all'utente attualmente autenticato (es. ID, Ruolo, Dipartimento) devono essere accessibili da molteplici Controller grafici in momenti diversi. Invece di passare continuamente l'oggetto utente come parametro da una vista all'altra il pattern Singleton garantisce un punto di accesso globale, sicuro e univoco allo "stato" dell'utente loggato, assicurando che tutta l'applicazione faccia riferimento alla medesima identità.

Nel diagramma delle classi ciò ha comportato: Entrambe le classi (DatabaseManager e Sessione) condividono la medesima firma strutturale del pattern: presentano un costruttore con visibilità private (per impedire l'istanziazione diretta dall'esterno mediante la keyword new). Entrambe definiscono un attributo privato e statico (es. private static Sessione instance;) che memorizza l'unica istanza consentita. Entrambe espongono un metodo pubblico e statico (es. public static Sessione getInstance()) che fornisce l'accesso globale all'istanza, creandola in modalità *lazy initialization* (solo alla prima reale richiesta) se non è già presente in memoria.

Observer: L'interfaccia grafica, essendo sviluppata con JavaFX, richiede un aggiornamento dinamico e reattivo in risposta ai cambiamenti dei dati sottostanti. Utilizzando strutture dati specifiche come ObservableList e il data binding tramite le Properties l'interfaccia (che agisce da *Observer*) si "abbona" e ascolta i cambiamenti dei dati (che agiscono da *Observable*), aggiornandosi automaticamente. Questo garantisce un basso accoppiamento (loose coupling) tra la vista e i dati, mantenendo i componenti indipendenti.

3.2 Gestione dei dati persistenti

Per la persistenza dei dati, l'applicazione si appoggia a un database relazionale. Come DBMS (Database Management System) è stato scelto MariaDB. Durante le fasi di sviluppo, il server database è stato ospitato localmente avvalendosi della suite XAMPP.

Struttura:

Il database è stato progettato normalizzando i dati per evitare ridondanze e garantire l'integrità referenziale. Tutte le relazioni sono configurate con vincoli di chiave esterna con la regola ON DELETE CASCADE, in modo da prevenire in modo assoluto la formazione di dati orfani nel sistema in caso di cancellazione.

Di seguito vengono descritte le entità principali del dominio:

1. **ateneo**: Memorizza i dati centrali dell'università (come l'indirizzo della sede centrale).
    - Chiave Primaria (PK): nome

2. **struttura**: Rappresenta i vari dipartimenti universitari.
    - Chiave Primaria (PK): nome
    - Chiavi Esterne (FK): ateneo_nome (collegata ad ateneo, con ON DELETE CASCADE).

3. **utente**: Contiene i dati di accesso (password, email) e le informazioni anagrafiche (nome, cognome, matricola).
    - Chiave Primaria (PK): matricola
    - Chiavi Esterne (FK): struttura_nome (collegata a struttura, con ON DELETE CASCADE).

4. **utente_ruolo**: Permette a un singolo utente di detenere più ruoli contemporaneamente.
    - Chiavi Primarie Composte (PK): utente_matricola, ruolo
    - Chiavi Esterne (FK): utente_matricola (collegata a utente, con ON DELETE CASCADE).

5. **incarico**: Tabella di associazione per storicizzare gli incarichi assunti dagli utenti all'interno delle varie strutture nel corso del tempo.
    - Chiavi Primarie Composte (PK): utente_matricola, struttura_nome, dataInizio
    - Chiavi Esterne (FK): Riferimenti a utente e struttura, entrambi con ON DELETE CASCADE.

6. **attivita**: Memorizza tutti i dati dei progetti proposti (titolo, periodo, stato attuale, tipologia). Per mappare l'ereditarietà delle classi Java all'interno del database relazionale è stato aggiunto un campo tipoAtt che discrimina la specializzazione dell'istanza, e i campi specifici di ciascuna specializzazione sono stati resi opzionali (NULLABLE).
    - Chiave Primaria (PK): id (Auto-increment)
    - Chiavi Esterne (FK): autore_matricola (collegata a utente, con ON DELETE CASCADE).

7. **allegato**: Memorizza i metadati dei documenti fisici caricati a sistema (percorso sul file system, dimensione, nome file originale).
    - Chiave Primaria (PK): id (Auto-increment)
    - Chiavi Esterne (FK): attivita_id (collegata ad attivita, con ON DELETE CASCADE).

8. **feedback**: Registra le valutazioni espresse sulle attività (esito positivo/negativo e note testuali).
    - Chiave Primaria (PK): id (Auto-increment)

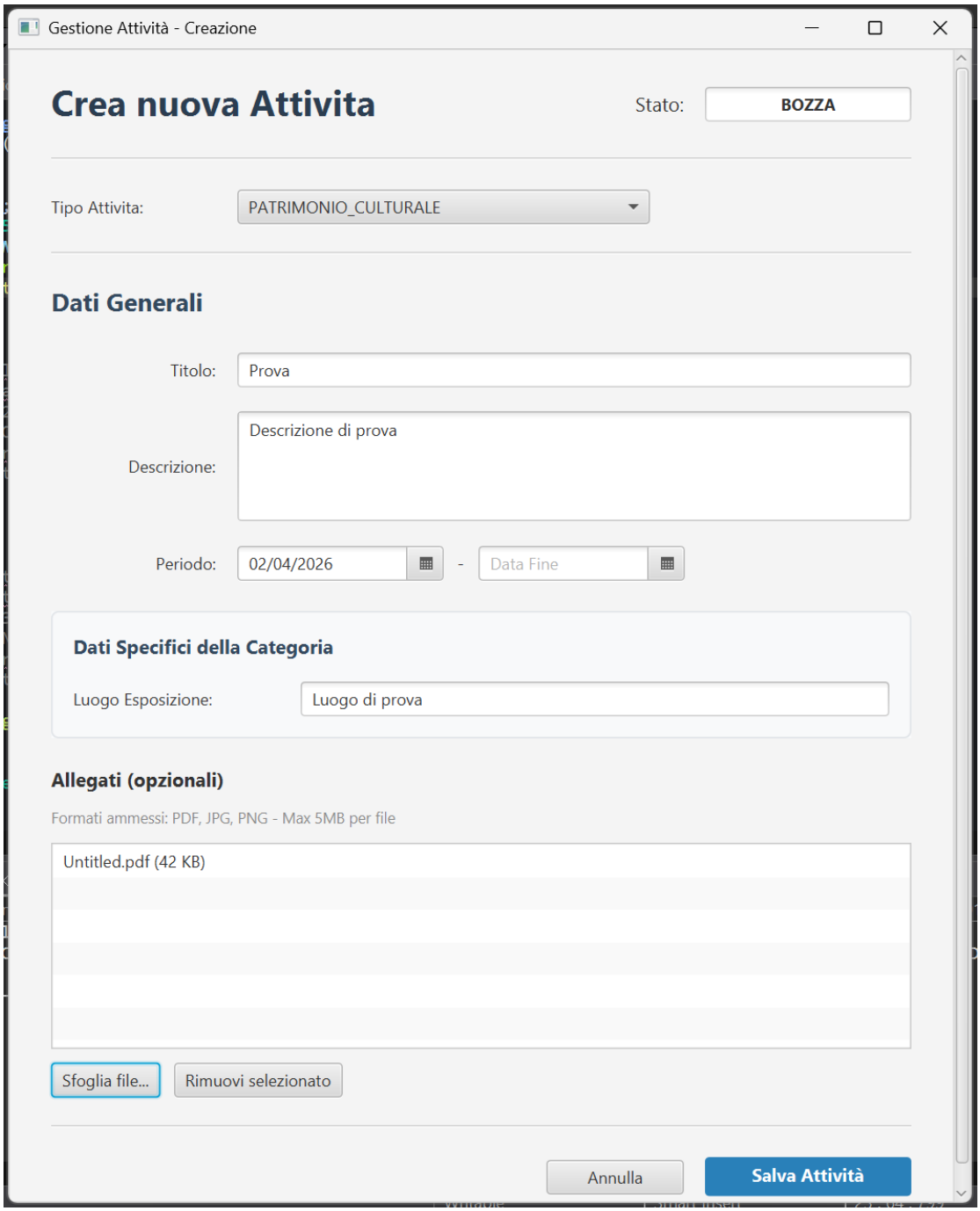
- Chiavi Esterne (FK): Riferimenti ad attivita (attivita_id) e all'autore della valutazione(autore_matricola), entrambi con ON DELETE CASCADE.

9. **notifica**: tiene traccia delle notifiche inviate agli utenti.
    - Chiave Primaria (PK): id (Auto-increment)
    - Chiavi Esterne (FK): Riferimenti al destinatario (destinatario_matricola) e all'attività (attivita_id), con ON DELETE CASCADE.

10. **report**: Salva i resoconti statistici generati dal sistema per consultazioni future.
    - Chiave Primaria (PK): id (Auto-increment)
    - Chiavi Esterne (FK): autore_matricola (collegata a utente, con ON DELETE CASCADE).

Per garantire la rigorosa coerenza temporale dei ruoli e alleggerire il carico computazionale del livello applicativo ho scelto di delegare la revoca dei ruoli scaduti direttamente al motore del database tramite l'implementazione di un event scheduler nativo. Questo evento chiamato pulizia_responsabili_scaduti viene eseguito in background con cadenza giornaliera e si occupa di verificare la validità temporale delle associazioni presenti nella tabella utente_ruolo. Per gestire correttamente i casi complessi in cui un utente abbia avuto più incarichi (alcuni scaduti e altri ancora in corso), l'algoritmo di pulizia adotta un approccio "ad esclusione" tramite una sotto-query (NOT IN). Il sistema rimuove il privilegio di 'RESPONSABILE' a un utente esclusivamente nel momento in cui nessuno dei suoi incarichi associati risulta più in corso di validità.

3.3 Descrizione dell'Interfaccia Grafica

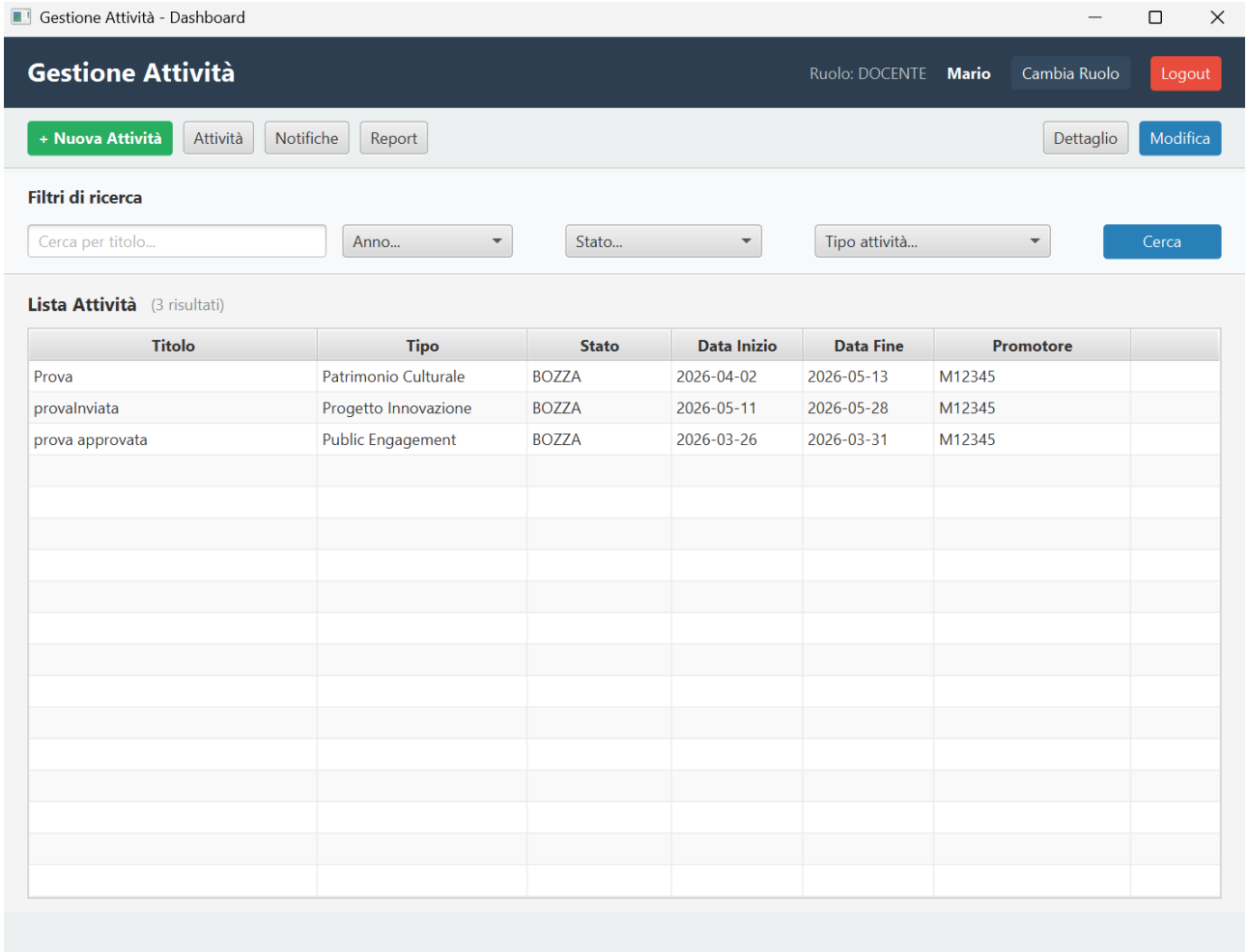


Controllo campi obbligatori form:
Prima di istanziare l'oggetto da salvare (nel metodo costruisciAttivitàDaForm()), il sistema verifica la completezza del form. I campi obbligatori per ciascun tipo di attività sono sottoposti ad un controllo di nullità o di consistenza. La Finalità è prevenire l'inserimento di record incompleti che violerebbero i vincoli NOT NULL del database, fornendo un feedback testuale immediato (lblErrore) senza far crashare l'applicativo

Controllo date:
Sempre durante la costruzione dell'oggetto, i valori prelevati dai DatePicker vengono sottoposti a due controlli temporali: che la data di inizio di un nuovo progetto non sia antecedente rispetto alla data odierna (dpDataInizio.getValue().isBefore(LocalDate.now())) e che la data di fine non sia prima della data di inizio(dpDataFine.getValue().isBefore(dpDataInizio.getValue())). La Finalità è applicare vincoli logici al dominio temporale dei progetti, garantendo la sensatezza dei dati salvati.

Controllo dell'estensione sui file allegati:
Nel metodo di acquisizione dei documenti (handleSfogliaFile()), l'oggetto FileChooser è stato configurato con un ExtensionFilter specifico (*.pdf, *.jpg, *.png). La finalità è effettuare una "whitelisting" preventiva (lato client) dei formati di file accettati, impedendo all'utente di selezionare e caricare eseguibili o formati non supportati dal sistema, riducendo i rischi di sicurezza legati all'upload
Prevenzione della Duplicazione degli Allegati:
Durante la selezione di nuovi file da allegare, il sistema verifica preventivamente che il file non sia già presente nella lista temporanea (!allegatiTemp.contains(file)). La Finalità è evitare il caricamento ridondante del medesimo file nello stesso slot di salvataggio, ottimizzando l'uso della memoria ed evitando errori di sovrascrittura in fase di persistenza locale.

Dashboard principale dell'applicativo con visibilità dei comandi condizionata dal ruolo attivo.



Nel metodo handleModifica() il Controller verifica in tempo reale lo stato dell'istanza selezionata prima di permettere la modifica. La sua finalità è garantire il rispetto di una fondamentale regola di business direttamente nella UI. Il sistema impedisce fisicamente la modifica di attività che non sono in stato di BOZZA

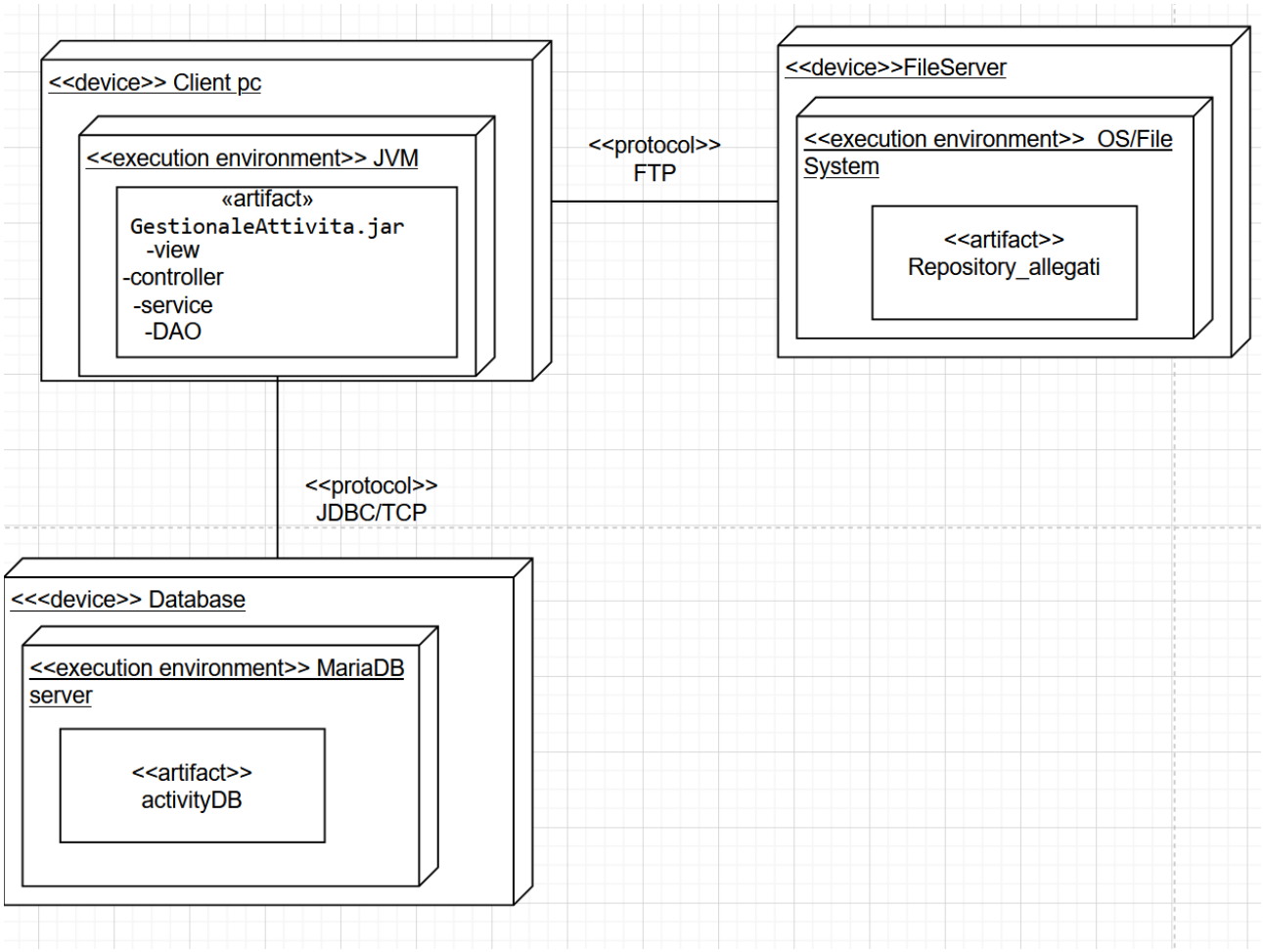
3.4 Diagramma di deployment

Il diagramma illustra l'architettura logicamente distribuita del sistema allo stato attuale (ad eccezione della semplificazione riguardante il salvataggio degli allegati, attualmente gestito in locale e non su un server FTP dedicato). Il nodo principale (Client pc) ospita l'ambiente di esecuzione della JVM e l'intero artefatto applicativo (GestionaleAttivita.jar), comprensivo dei livelli di Presentazione

(View), Logica di Business (Controller, Service) e Accesso ai Dati (DAO). Il Client comunica tramite protocolli di rete con due nodi remoti dedicati:

1. **Database Server:** Ospita il RDBMS MariaDB per la persistenza.
2. **File Server:** Nodo concettuale adibito allo storage degli allegati (attualmente simulato sul file system locale ai fini della prototipazione).

Evoluzione Architetturale: Sebbene l'attuale implementazione soddisfi i requisiti prototipali, il sistema è progettato per supportare una naturale evoluzione verso una pura architettura a 3 livelli. In uno scenario di produzione su larga scala, l'architettura prevede lo spostamento dei package Service e DAO su un nodo intermedio dedicato (Application Server). Quest'ultimo esporrà la logica di dominio tramite una serie di API. Il Client verrebbe così alleggerito mantenendo esclusivamente la View e i relativi Controller di Interfaccia.



3.5 Specifica e verifica dei vincoli

Vincolo 1: Coerenza Temporale

In qualsiasi istanza di attività, la data di fine non può essere prima della data di inizio.

{ context Attività inv DateValide: self.dataFine >= self.dataInizio }

Viene tradotto in JML all'interno della classe attività:

//@ invariant dataFine == null || dataInizio == null || dataFine.compareTo(dataInizio) >= 0;

Vincolo 2: Vincolo di Modifica (Requisito RF7)

Un'attività può essere modificata solo se si trova nello stato di BOZZA.

```
{ context GestioneAttivitaService::modificaAttivita(attivita: Attivita, nuoviFile: List<File>,
allegatiDaRimuovere: List<Integer>) pre ModificabileSoloInBozza: attivita.stato =
StatoAttivita::BOZZA }
```

Viene tradotto in JML all'interno della classe GestioneAttivitàService:

```
/*@  requires attivita != null && attivita.getStato() == StatoAttività.BOZZA; @*/
```

Vincolo 3: Conflitto d'Interessi e Stato nell'Approvazione (Requisiti RF4 e RF4.2)

L'approvazione richiede che l'attività sia nello stato corretto, che l'utente sia un Responsabile e che l'approvatore non coincida con l'autore. Come post si ottiene che l'attività si trova nello stato APPROVATA

```
{ context WorkflowService::approvaAttivita(attivitaId: int, utente: Utente)
  pre StatoCorrettoPerApprovazione:
     Attivita.allInstances()->select(a | a.ID = attivitaId)->first().stato = StatoAttivita::INVIATA
  pre RuoloAutorizzato: utente.ruolo = Ruolo::RESPONSABILE
  pre SegregazioneRuoli:
     Attivita.allInstances()->select(a | a.ID = attivitaId)->first().autore <> utente
  post CambioStato:
     Attivita.allInstances()->select(a | a.ID = attivitaId)->first().stato = StatoAttivita::APPROVATA }
```

Vincolo 4 Regole per la Validazione

La validazione finale può essere eseguita solo da un Amministratore su un'attività precedentemente approvata

```
{ context WorkflowService::validaAttivita(attivitaId: int, utente: Utente)
  pre StatoCorrettoPerValidazione:
     Attivita.allInstances()->select(a | a.ID = attivitaId)->first().stato = StatoAttivita::APPROVATA
  pre RuoloAutorizzato: utente.ruolo = Ruolo::AMMINISTRATORE
  post CambioStatoDefinitivo:
     Attivita.allInstances()->select(a | a.ID = attivitaId)->first().stato = StatoAttivita::VALIDATA }
```

Vincolo 5: Regole per il rifiuto

Un'attività inviata o approvata può essere respinta e riportata in bozza da Responsabili o Amministratori a condizione che venga fornita obbligatoriamente una motivazione testuale.

```
{context WorkflowService::rifiutaAttivita(attivitaId: int, utente: Utente, motivazione: String)
pre StatoERuoloCoerenti:
 (utente.ruolo = Ruolo::RESPONSABILE and Attivita.allInstances()->select(a | a.ID = attivitaId)->
first().stato = StatoAttivita::INVIATA)
   or
 (utente.ruolo = Ruolo::AMMINISTRATORE and Attivita.allInstances()->select(a | a.ID = attivitaId)->
first().stato = StatoAttivita::APPROVATA)

pre MotivazioneObbligatoria:
   motivazione <> null and motivazione.size() > 0

post RetrocessioneBozza:
   Attivita.allInstances()->select(a | a.ID = attivitaId)->first().stato = StatoAttivita::BOZZA
}
```

3.6 Descrizione del testing

La fase di testing automatizzato si è concentrata sul livello di persistenza dei dati (Data Access Object), in particolare sulla classe AttivitàDAOIMPL. Il criterio di selezione dei casi di test adottato è la Copertura dei Rami (Branch Coverage). Per garantire l'isolamento dei test la classe di test implementa metodi di setup e di pulizia finale. @BeforeAll: Inizializza una connessione al database e genera un utente fittizio per soddisfare i vincoli di chiave esterna (Foreign Key) richiesti per l'inserimento delle attività. @AfterEach: Svuota le tabelle ripristinando lo stato del database.

Casi di Test Implementati:

Copertura del Polimorfismo (Branch instanceof): Sono stati implementati tre test distinti (testCreatePatrimonioCulturale, testCreateProgettoInnovazione, testCreatePublicEngagement) per verificare che il DAO instradi correttamente i dati specifici di ogni sottoclasse nelle rispettive colonne del database relazionale.

Copertura delle Entità Deboli (Branch Liste Vuote/Piene): Attraverso i test testCreateConAllegati e testCreateSenzaAllegati, si certifica il corretto attraversamento del blocco condizionale dedicato al salvataggio a cascata dei file associati all'attività

Copertura dei Fallimenti e Vincoli di Integrità: Un test apposito (es. simulando un inserimento con un utente inesistente) verifica il comportamento del sistema in caso di fallimento della query INSERT, assicurandosi che l'eccezione SQL venga gestita correttamente e che l'ID dell'oggetto Java non venga illecitamente aggiornato in caso di transazione non andata a buon fine.

3.7 Note per l'installazione e l'utilizzo

Per testare le funzionalità di upload e gestione documenti (Crea/Modifica Bozza), è necessario assicurarsi che l'utente che esegue l'applicazione disponga dei **permessi di lettura e scrittura** sulla directory principale del progetto.

L'applicazione è stata sviluppata e strutturata utilizzando:

Linguaggio e UI: Java (JavaSE-21) con il framework JavaFX per la realizzazione dell'interfaccia utente grafica.

IDE: Eclipse

Strumento di Build: Maven. l'importazione delle librerie esterne (driver JDBC per MariaDB, moduli JavaFX, librerie JUnit 5 per il testing) sono orchestrate tramite il file pom.xml.

Database: MariaDB. Durante lo sviluppo, il server database è stato ospitato localmente sfruttando il pacchetto software XAMPP.

Per la corretta esecuzione dell'applicativo in fase di collaudo, è necessario disporre di un'istanza locale di MariaDB e importare lo script SQL di dump fornito in allegato al progetto.

I parametri di connessione JDBC sono stati inglobati direttamente in un'unica stringa (URL di connessione) all'interno della classe DatabaseManager

URL di Connessione: jdbc:mariadb://localhost/progettomd?user=root

Host: localhost

Porta: 3306 (implicita di default per MariaDB/MySQL)

Nome Database: progettomd

Username: root

Password: nessuna

Simulazione della Sessione e Utenti di Test

Per facilitare le fasi di sviluppo, collaudo e verifica dell'interfaccia, è stato implementato un meccanismo di iniezione di "utenti di test" (mock) all'avvio dell'applicazione. Come evidenziato dal codice, istanziando manualmente un oggetto Utente e forzandone l'inserimento nel Singleton della Sessione, è possibile simulare l'accesso con diversi livelli di privilegio (Docente, Responsabile, Amministratore). Commentando o decommentando i relativi blocchi è possibile testare i diversi livelli di privilegio.

Profilo: DOCENTE

- **Nome e Cognome:** Mario Rossi
- **Matricola:** M12345
- **Struttura di appartenenza:** informatica

Profilo: RESPONSABILE

- **Nome e Cognome:** responsabile responsabile
- **Matricola:** MAT002
- **Struttura di appartenenza:** informatica

Profilo: AMMINISTRATORE

- **Nome e Cognome:** amministratore amministratore
- **Matricola:** MAT003
- **Struttura di appartenenza:** amministrazione